

SMART ENERGY METER WITH REALTIME-MONITORING AND PREDICTIVE BILL ANALYSIS

A PROJECT REPORT ON

*Submitted in partial fulfillment of the requirements
for the award of the degree of*

BACHELOR OF TECHNOLOGY

in

ELECTRONICS AND COMMUNICATION ENGINEERING

Submitted by

D. NAGA JASWANTH

22JR5A0406

E. P. L. NARASIMHA

21JR1A0459

B. AAKASH

21JR1A0436

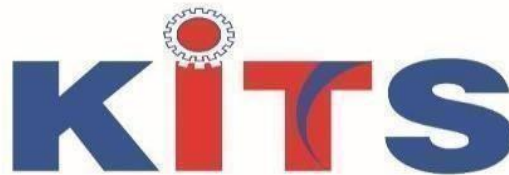
B. ASHOK BABU

21JR1A0446

Under the Guidance of

Mrs. SD. VAHEEDHA, M. Tech

Assistant Professor



(AUTONOMOUS)

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
KKR & KSR INSTITUTE OF TECHNOLOGY AND SCIENCES
(AUTONOMOUS)**

Vinjanampadu (V), Vatticherukuru (M), Guntur -522017

**JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY KAKINADA: KAKINADA
APRIL-2025**



CERTIFICATE

This is to certify that this project work entitled “**SMART ENERGY METER WITH REALTIME-MONITORING AND PREDICTIVE BILL ANALYSIS**” is a Bonafide work of **Mr. D. NAGA JASWANTH (22JR5A0416), Mr. E. P. L. NARASIMHA (21JR1A0459), Mr. B. AAKASH(21JR1A0436), Mr. B. ASHOK KUMAR (21JR1A0446)** to Jawaharlal Nehru University Kakinada, through KKR & KSR Institute of Technology and Sciences for the award of the Degree of Bachelor of Technology in **Electronics and Communication Engineering** is a bonafide record of project work carried out by **Mrs. SD. Vaheedha** under my supervision during the year 2021-2025.

SUPERVISOR

Mrs. SD. VAHEEDHA, M. Tech.
Assistant Professor

Head of Department ECE

Dr. N. Adi Narayana, M.Tech, Ph.D

INTERNAL EXAMINAR

EXTERNAL EXAMINAR

DECLARATION

We hereby declare that the project “**SMART ENERGY METER WITH REALTIME-MONITORING AND PREDICTIVE BILL ANALYSIS**” has been carried out by me and this work has been submitted to KKR & KSR Institute of Technology and Sciences (A), Vinjanampadu, affiliated to Jawaharlal Nehru Technological University, Kakinada in partial fulfillment of the requirements for the award of degree of Bachelor of Technology in Department Of Electronics And Communication Engineering.

We further declare that this project work has not been submitted in full or part for the award of any other degree in any other educational institutions.

1. 22JR5A0406 DAMARAVARAPU NAGA JASWANTH - Signature

2. 21JR1A0459 EKA P. L. NARASIMHA - Signature

3. 21JR1A0436 BANISETTI AAKASH - Signature

4. Roll Number BOLLU ASHOK BABU - Signature

ACKNOWLEDGEMENT

We express our sincere gratitude and indebtedness to **Mr. K. Subba Rao**, Chairman and **Mr. K. Sekhar**, Secretary of the esteemed institution **KKR and KSR INSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)** for providing us with all the necessary support.

We render my deep sense of gratitude of **Dr. P. BABU**, the Principal for permitting us to carry out our project work. I would also like to thank the **ELECTRONICS AND COMMUNICATION ENGINEERING** staff for lending me their time to help us and to complete our work successfully.

We render our deep sense of gratitude to **Dr. K. HARI BABU** Director (academics) for permitting us to carry out our project work. We are highly indebted to **Dr. N. Adi Narayana**, Head of the Department of **ELECTRONICS AND COMMUNICATION ENGINEERING** for providing us with all the necessary support.

We would like to express our gratitude towards **Mrs. SD. VAHEEDHA, M. Tech, Assistant Professor**, DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING, who played a supervisory role to the utmost perfection, enabled us to seek our main project and for guiding with an internal guide methodically and meticulously.

We express **our wholehearted thanks** to our PROJECT CO-ORDINATOR **Dr. M. RAMBABU NAIK**, friends and staff for their wishes and for helping in the successful completion of this project.

By

D. NAGA JASWANTH

22JR5A0406

E. P. L. NARASIMHA

21JR1A0459

B. AAKASH

21JR1A0436

B. ASHOK BABU

21JR1A0446

KKR & KSR INSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)

(Accredited by NAAC with 'A' grade, Approved by AICTE, New Delhi & Affiliated to JNTU Kakinada)

INSTITUTION VISION

To produce eminent and ethical Engineers and Managers for society through imparting quality professional education with an emphasis on human values and holistic excellence.

INSTITUTION MISSION

- To incorporate benchmarked teaching and learning pedagogies in the curriculum.
- To ensure the all-round development of students through a judicious blend of curricular,
Co-curricular and extracurricular activities.
- To support the cross-cultural exchange of knowledge between industry and academy.
- To provide higher/continued education and research opportunities to the employees of the institution.

DEPARTMENT OF ECE

VISION

Developing highly qualitative, technically competent and socially responsible engineers.

MISSION

To provide quality education in the domain of Electronics and Communication Engineering through.

- Enriched curriculum for addressing the needs of Industry.

KKR & KSR INSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)

(Accredited by NAAC with 'A' grade, Approved by AICTE, New Delhi & Affiliated to JNTU Kakinada)

- Effective teaching and learning processes through the congenial environment.

Gaining contemporary knowledge through research, development, and co-curricular activities.

POs & PEOs

The Department has a formally stated quality policy which is being implemented through a well-structured body called Internal Quality Assurance Cell. Since quality enhancement is a continuous process, the IQAC is a part of the system and works towards realizing the goals of quality enhancement and sustenance.

Program Outcomes (PO's)

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

KKR & KSR INSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)

(Accredited by NAAC with 'A' grade, Approved by AICTE, New Delhi & Affiliated to JNTU Kakinada)

6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSO's)

PSO1

Professional Skills:

To understand the basic concepts of Electronics & Communication Engineering and to apply them to various areas, like Electronics, Communications, VLSI, and Embedded systems to arrive cost-effective and appropriate solutions.

KKR & KSR INSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)

**(Accredited by NAAC with 'A' grade, Approved by AICTE, New Delhi & Affiliated to JNTU Kakinada)
PSO2**

Problem-solving skills:

To be equipped with the principles of Semiconductor Devices, Digital Systems, Microprocessor and Signal processing for the fields of Consumer Electronics, Medical, Defense and Spacecraft Electronics industry

PSO3

Successful career:

To be successful in professional career by using latest hardware and software tools like VHDL, MATLAB, MULTISIM, MENTOR GRAPHICS along with analytical skills to work in core industries

PSO4

Exposure for research and development:

To analyze latest trends in Electronics and Communication and apply the knowledge for the improvement in the present technology by doing research through higher education.

ECE Program Educational Objectives

Graduates of Electronics & Communication Engineering Shall

PEO1

Develop a strong background in basic science and mathematics and ability to use these tools in their chosen fields of specialization.

PEO2

**KKR & KSR INSTITUTE OF TECHNOLOGY AND SCIENCES
(AUTONOMOUS)**

(Accredited by NAAC with 'A' grade, Approved by AICTE, New Delhi & Affiliated to JNTU Kakinada)

Have the ability to demonstrate technical competence in the fields of electronics and communication engineering and develop solutions to the problems.

PEO3

Attain professional competence through life-long learning such as advanced degrees, professional registration, and other professional activities.

PEO4

Function effectively in a multi-disciplinary environment and individually, within a global, societal, and environmental context.

PEO5

Take individual responsibility and to work as a part of a team towards the fulfillment of both individual and organizational goals.

KKR & KSR INSTITUTE OF TECHNOLOGY AND SCIENCES (AUTONOMOUS)

(Accredited by NAAC with 'A' grade, Approved by AICTE, New Delhi & Affiliated to JNTU Kakinada)

COURSE OUTCOMES

C 409.1	Design and develop an IoT-based Smart Energy Meter system capable of real-time monitoring of voltage, current, power, and energy consumption.
C 409.2	Implement predictive algorithms to estimate monthly electricity bills based on real-time energy usage data.
C 409.3	Integrate cloud technologies Thingspeak and mobile platforms (Blynk) for remote data visualization, storage, and device control.
C 409.4	Develop automation mechanisms for household electrical loads using relay modules, enabling remote switching and energy optimization.
C 409.5	Generate real-time alerts and notifications for high voltage events, threshold breaches, and abnormal energy consumption to enhance system reliability and safety..
C 409.6	Analyze and evaluate the performance of the Smart Energy Meter system to ensure energy efficiency, data accuracy, and cost-effectiveness in real-world environments.

Course Outcomes - Co-Po Mapping

	PO 1	PO 2	PO 3	PO 4	PO 5	PO 6	PO 7	PO 8	PO 9	PO10	PO11	PO12	PSO1	PSO2	PSO3	PSO4
CO421.1	3		2													
CO421.2																
CO421.3																
CO421.4																
CO421.5																
CO421.6																

3: High 2: Medium 1: Low

KKR & KSR INSTITUTE OF TECHNOLOGY AND SCIENCES
(AUTONOMOUS)

(Accredited by NAAC with 'A' grade, Approved by AICTE, New Delhi & Affiliated to JNTU Kakinada)

**Program Educational Objectives – Program Specific Outcomes
correlation**

	PSO1	PSO2	PSO3	PSO4
PEO1				
PEO2				
PEO3				
PEO4				
PEO5				

3: High 2: Medium 1: Low

INDEX

CONTENTS	PAGE NO.
ABSTRACT	i
CHAPTER 1: INTRODUCTION	
1.1 Overview	
1.2 Objective	
1.3 Organization Of the Thesis	
CHAPTER 2 : Literature Survey	
2.1 Literature Survey	
CHAPTER 3 :Existing Work with references	
CHAPTER 4: Proposed work	
4.1 Introduction	
4.2 Block Diagram	
4.3 Working Procedure	
4.4 Flow Chart	
4.5 Hardware Components	
4.6 Description of Major Components	
4.6.1 ESP32 Microcontroller	
4.6.1.1 Features and specifications	
4.6.1.2 GPIO pin configuration	
4.6.1.3 Communication interfaces (I2C, UART, Wi-Fi)	
4.6.1.4 Role in the energy meter system	
4.6.2 PZEM-004T Energy Meter	
4.6.2.1 Working principle	
4.6.2.2 Electrical parameters	
4.6.2.3 Connection with ESP32	
4.6.2.4 Communication protocol (UART)	
4.6.3 ACS712 Current Sensor	
4.6.3.1 Overview of ACS712	
4.6.3.2 Measuring Individual loads	
4.6.3.3 Sensitivity and output voltage relation	

- 4.6.3.4 Interfacing with ESP32
 - 4.6.3.5 Calculation of current and power
 - 4.6.4 4-channel Relay Module
 - 4.6.4.1 Relay Module working principle
 - 4.6.4.2 Operating Voltages and currents
 - 4.6.4.3 Controlling of Individual Loads
 - 4.6.4.4 Interfacing with ESP32
 - 4.6.5 DHT11 Temperature Sensor
 - 4.6.5.1 Features and Specifications
 - 4.6.5.2 Temperature and humidity sensing
 - 4.6.5.3 Connection and data from ESP32
 - 4.6.6 16x2 I2C Display
 - 4.6.6.1 LCD Display Specifications
 - 4.6.6.2 I2C communication protocol
 - 4.6.6.3 Interfacing with ESP32
 - 4.6.7 Power supply
 - 4.6.7.1 Power requirements for ESP32
 - 4.6.7.2 5V and 3.3V voltage regulation
 - 4.6.7.3 Power supply circuit design
 - 4.7.8 Arduino IDE 2.3.4 Setup
 - 4.7.8.1 Introduction
 - 4.7.8.2 Install Arduino IDE
 - 4.7.9 Blynk Application
 - 4.7.9.1 Introductions and
 - 4.7.9.2 Setup
 - 4.7.10 Thinkspeak
 - 4.7.10.1 Introduction
 - 4.7.10.2 Setup
- 4.7 Project Code

CHAPTER 5: Testing and Result Analysis

5.1 TestCase1

5.2 TestCase2

5.3 TestCase3

5.4 Test case4

5.5 Test Case5

CHAPTER 6: Conclusions and Scope for Future work

6.1 Conclusion

6.2 Future Scope

ABSTRACT

In With the increasing demand for efficient energy management, traditional electricity meter fails to provide real-time insights into power consumption. This project presents the design and implementation of a Smart Energy Meter with Real-Time Monitoring and predictive bill analysis using an ESP32 microcontroller. The system integrates a PZEM-004T energy meter for total power measurements, ACS712 current sensors for individual load monitoring, and a DHT11 sensor for environmental parameters. A 4-channel relay module allows remote load control, while a **16x2 I2C LCD** display provides real-time energy data and also get the data in Thingspeak platform and we will download in .csv file format.

The Smart meter continuously measures voltage, current, power and energy consumption and calculates predictive billing based on past trends. The data is displayed locally and transmitted to the Blynk IoT platform, enabling users to monitor energy usage and control loads remotely. The proposed design enhances consumer awareness, improved energy efficiency and offers a cost-effective solution for Home Automation

Keywords— Smart Energy Meter, ESP32, PZEM-004T, ACS712, Real-time Monitoring, Predictive Bill Analysis, IoT, Home Automation, Blynk.

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

1.1 Overview

Electricity consumption has become an essential aspect of modern life, with increasing demand in residential, commercial, and industrial sectors. Traditional energy meters, which rely on manual readings and post-consumption billing, often lack real-time monitoring and predictive analysis capabilities. This limitation results in inefficient energy usage, unexpected high electricity bills, and a lack of user awareness regarding energy consumption patterns.

To overcome these challenges, this project introduces a **Smart Energy Meter** capable of **real-time energy monitoring, predictive bill analysis, and remote load control** using IoT technology. The proposed system utilizes an **ESP32 microcontroller** interfaced with a **PZEM-004T energy meter** for total power measurement and **ACS712 current sensors** to monitor individual loads. Additionally, a **DHT11 sensor** is integrated to measure environmental parameters such as temperature and humidity. A **4-channel relay module** is used to control appliances, while a **16x2 I2C LCD display** provides real-time feedback.

By leveraging IoT and cloud-based solutions, the system sends energy consumption data to the **Blynk platform**, enabling users to access power usage statistics remotely. The **predictive bill analysis** feature estimates future electricity costs based on past consumption patterns, helping consumers make informed decisions about energy usage. Furthermore, users can set predefined consumption thresholds, receiving alerts when energy usage exceeds expected limits. This **proactive approach to energy management** enhances efficiency, reduces costs, and promotes sustainable electricity consumption.

1.2 Objectives

The primary objective of this project is to design and implement a smart energy meter that enables users to monitor and control their electricity usage effectively. The specific goals of the system are as follows:

1. **Real-Time Energy Monitoring:** Measure voltage, current, power, and energy consumption using PZEM-004T and ACS712 sensors.
2. **Predictive Bill Analysis:** Calculate electricity costs based on past trends to help users anticipate their monthly bills.
3. **Remote Load Control:** Enable users to turn appliances on or off remotely via the Blynk platform.
4. **User-Friendly Display:** Provide real-time readings on a 16x2 I2C LCD display for easy on-site monitoring.
5. **Threshold-Based Alerts:** Notify users when energy consumption exceeds predefined limits.
6. **Home Automation Integration:** Allow smart control of electrical loads through relays for better energy management.

By achieving these objectives, the smart energy meter ensures transparency in electricity consumption, encourages efficient usage, and provides a cost-effective solution for modern energy monitoring.

1.3 Motivation and Need for Smart Energy Meters

The growing global focus on **energy efficiency, sustainability, and cost reduction** has highlighted the need for smarter electricity management solutions. Traditional meters only provide cumulative energy usage without real-time tracking, making it difficult for users to optimize their consumption patterns.

In many households and businesses, **unexpected high electricity bills** are a common issue. Users often do not realize their excessive energy usage until they receive their monthly bill. A smart energy meter bridges this gap by **continuously monitoring power consumption** and providing **real-time alerts and insights**,

enabling users to make informed energy-saving decisions.

Moreover, the rise of **home automation** and **IoT-based smart systems** has led to increased demand for **remote monitoring and control solutions**. A smart energy meter with **IoT integration** not only tracks consumption but also allows users to **remotely control appliances**, reducing unnecessary power wastage.

Additionally, industries and large-scale consumers often experience **power losses due to inefficient load management**. By implementing a predictive bill analysis system, businesses can **estimate electricity expenses in advance** and optimize their operations accordingly. This approach also contributes to **sustainable energy practices** by encouraging users to adopt **energy-efficient appliances** and reduce excessive consumption.

1.4 Organization of the Report

This report is structured into multiple chapters, each covering different aspects of the smart energy meter project:

- **Chapter 1: Introduction** – Provides an overview of the project, objectives, motivation, and necessity of smart energy meters.
- **Chapter 2: Literature Survey & System Architecture** – Discusses existing energy metering systems, their limitations, and the proposed solution.
- **Chapter 3: Methodology** – Explains the block diagram, working principles, hardware integration, and software implementation.
- **Chapter 4: Hardware Components** – Covers a detailed description of all hardware components used in the project.
- **Chapter 5: Software Implementation** – Explains the software development process, including coding, data processing, and remote control features.
- **Chapter 6: Results and Discussion** – Analyzes the performance of the system based on real-time testing and observations.
- **Chapter 7: Conclusion and Future Enhancements** – Summarizes the key findings and suggests future improvements for the system.

CHAPTER 2

2.1 Literature survey

1. Smart Energy Management and Load Monitoring of Individual Loads

The work by Mohamed Aashik et al. presents an IoT-based energy management system targeting the measurement and monitoring of energy consumption of individual household appliances. Unlike conventional meters that provide only the total consumption, their proposed system measures the power used by each device separately using ESP32, voltage, and current sensors. Data is processed and displayed through a web application, enabling users to detect anomalies and forecast equipment lifespan based on usage.

The authors highlight the limitations of existing energy meters, such as the lack of individual load monitoring, cost calculations, remote control, and malfunction alerts. To address these, the proposed system uses a voltage sensor (ZMPT101B) and current sensor (SCT-013) along with relay modules for load control. Real-time statistics like energy consumption, cost estimation, and load behavior are stored and visualized through a mobile-friendly web interface, enhancing user engagement and energy-saving awareness.

The study emphasizes the critical need for energy conservation, especially given the increasing energy consumption trends in India's residential and commercial sectors. The proposed prototype allows users to control appliances remotely, monitor energy usage continuously, and predict device health — thus paving the way for efficient and intelligent energy management systems.

2. Energy Meter Based Wireless Monitoring System Using Blynk Application

Ammar Othman and Nur Hafiza Zakaria developed a wireless energy monitoring system using the Blynk application and ESP32, aimed at simplifying electricity usage tracking for users. The system addresses issues faced during the COVID-19 pandemic when manual meter reading was suspended, providing a reliable solution through IoT and smartphone integration. By sending real-time energy consumption data to a mobile app, the system enhances user trust in digital meters and promotes energy-saving habits via notifications.

The research introduces the concept of integrating backup battery systems to ensure continuous operation even during power outages. The system calculates RMS voltage and current, power, and energy consumption using AC sensors. Moreover, the Blynk application provides user-friendly visualization of energy metrics and notifications for over-consumption, thus making users more aware of their energy patterns and encouraging responsible usage.

In the future, the authors propose embedding Artificial Intelligence (AI) into such energy management systems. AI can automate decision-making processes, improving load management efficiency in smart homes. The paper lays the groundwork for smarter, more resilient energy monitoring solutions that combine IoT with intelligent analytics for optimal energy utilization.

3. ESP32 Based Electric Energy Consumption Meter

Paul Stone Macheso and Doreen Thotho propose a cost-effective smart energy monitoring solution using ESP32, SCT-013 current sensors, and ZMPT101B voltage sensors. Their system collects real-time energy consumption data and uploads it to the Blynk server, enabling users to monitor key parameters like voltage, current, active power, and energy consumption remotely. The IoT-based architecture ensures that even traditional digital meters can be enhanced for real-time monitoring through cloud connectivity.

The paper reviews existing energy monitoring systems, citing solutions based on GSM, Bluetooth, and Wi-Fi communications. It critiques their limitations, such as high costs and limited data accessibility. By leveraging the ESP32's Wi-Fi capabilities, the proposed system reduces complexity and improves reliability. The use of non-invasive sensors ensures ease of installation without modifying existing electrical wiring.

Additionally, the authors demonstrate the practical benefits of their system, including detecting abnormal energy consumption patterns, forecasting electricity costs, and providing actionable insights to consumers. Their approach significantly contributes to the global move toward smart grids and energy-efficient homes by offering an affordable and easily deployable solution.

4. IoT-Based Smart Energy Meter

Sanket Tule et al. present an IoT-based smart energy meter that automates energy monitoring and billing using the HLW8012 sensor and ESP32 microcontroller. The system continuously tracks consumption and updates the total bill in real-time through a smartphone application. This automation reduces human error, labor costs, and delays associated with traditional manual reading methods, offering a seamless and transparent billing system for both consumers and suppliers.

The proposed design uses the Virtuino app for mobile visualization, displaying key parameters like voltage, current, power, and total units consumed. Distance switching of loads is made possible, empowering users to control their home appliances remotely. The system also supports multiple user logins (for users and administrators), enhancing management efficiency and security.

By incorporating cloud communication, the smart meter prototype provides a future-ready platform that can be expanded into prepaid metering models. This development is particularly significant for electricity distribution companies in regions like India, where manual errors and tampering issues are prevalent, thereby modernizing energy infrastructure with minimal operational disruptions.

5. Design of a Smart Energy Meter

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

Niloy Mondal and Ravisankar Ch propose an IoT-based smart energy meter design utilizing ESP32, ZMPT101B voltage sensors, and ACS712 current sensors. Their system enables real-time monitoring of household energy consumption, with data displayed on a 16x2 LCD and sent to a mobile application via the Blynk platform. The integrated relay module allows users to remotely control electrical loads, enhancing the interactivity of the energy management system.

The authors focus on improving transparency and reducing corruption associated with manual meter reading processes. By automating meter readings and bill generation, their design empowers consumers to view detailed breakdowns of daily, monthly, and yearly energy usage. The system also implements safety features like load disconnection in case of overcurrent conditions, ensuring appliance protection and consumer safety.

Through cloud integration, the proposed system provides detailed, graphical data analysis, offering users actionable insights into their consumption patterns. The ability to control loads remotely via a smartphone app, combined with real-time monitoring, positions this smart energy meter as a critical advancement towards achieving efficient and user-friendly smart home energy management.

CHAPTER 3

Existing System

Energy consumption monitoring has traditionally relied on electromechanical energy meters, which have been widely used in residential, commercial, and industrial sectors. These meters operate based on an induction mechanism, where a rotating aluminum disk moves proportionally to the amount of energy consumed. The cumulative energy usage is displayed on a mechanical counter, requiring manual reading by utility personnel at regular intervals. While these meters have been reliable for decades, they suffer from several limitations, including the lack of real-time monitoring, remote access, and automation. Additionally, human errors in manual meter readings often lead to billing discrepancies, causing inconvenience for both consumers and utility providers.



Fig. Arduino UNO



Fig. GSM Module

To address the shortcomings of electromechanical meters, digital energy meters were introduced. These meters incorporate microcontrollers and electronic sensors to measure voltage, current, and power more accurately. They provide electronic displays that eliminate the need for mechanical counters and enable automated data storage. Some digital meters also include tamper detection mechanisms to prevent fraudulent

activities and prepaid metering options, allowing consumers to recharge their electricity balance in advance. However, despite these improvements, digital meters still do not provide real-time consumption insights or remote monitoring capabilities, leaving consumers unaware of their energy usage patterns until they receive their monthly bills.

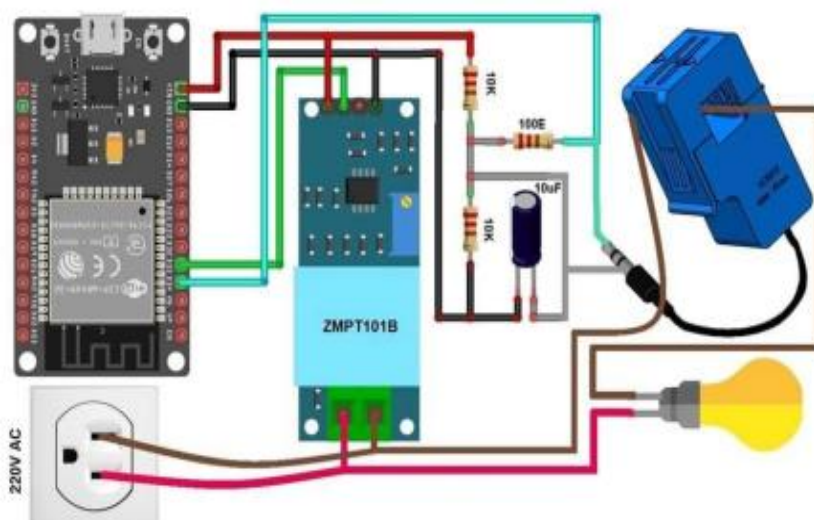
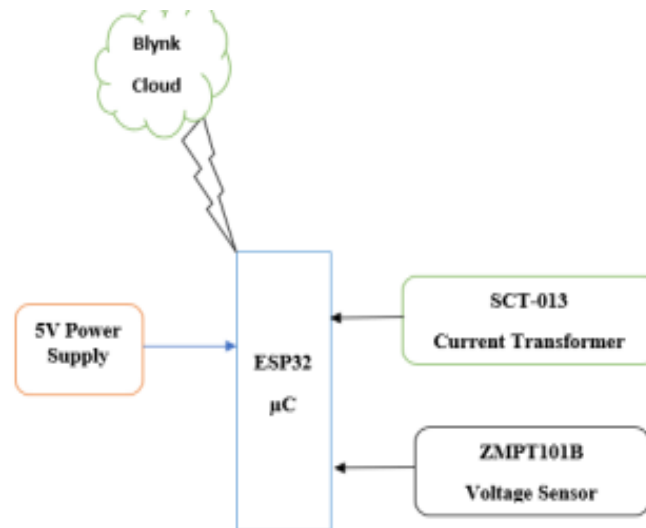
With advancements in Internet of Things (IoT) technology, smart energy meters have emerged as a more efficient alternative to conventional energy meters. These meters integrate wireless communication technologies such as Wi-Fi, GSM, ZigBee, or Bluetooth, allowing real-time data transmission to cloud-based platforms. IoT-enabled meters use microcontrollers like ESP32, Raspberry Pi, or Arduino, along with sensors such as PZEM-004T for total energy measurement and ACS712 for individual load current sensing. The integration of cloud computing enables users to remotely access their energy consumption data through mobile applications, providing greater transparency and control over power usage.

Despite the advantages of IoT-based smart meters, existing systems still have limitations. Most smart meters primarily focus on total energy consumption monitoring rather than tracking individual appliances separately. This lack of granularity prevents users from identifying which specific loads contribute the most to their electricity bills. Furthermore, many smart meters do not include predictive billing analysis, meaning users cannot estimate their future electricity costs based on their current usage trends. This absence of forecasting capabilities makes it difficult for consumers to plan and optimize their energy consumption effectively.

Another critical drawback of the existing smart metering systems is the delayed billing cycle, which prevents proactive energy management. Consumers only receive their electricity bills at the end of the billing period, leading to unexpected high charges and limited opportunities for corrective actions. Additionally, most systems do not provide threshold-based alerts, which means consumers remain unaware when their power consumption exceeds predefined limits. Without such alerts, users cannot take preventive measures to reduce excessive energy usage, potentially leading to higher costs and inefficiencies.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

Moreover, many existing smart meters lack remote load control functionalities, limiting the ability to turn appliances on or off from a remote location. Home automation features are becoming increasingly important in modern energy management systems, as they allow users to optimize electricity usage by controlling appliances via mobile applications. However, most current solutions focus only on monitoring and reporting, rather than providing an interactive system that allows users to take real-time actions based on energy consumption data.



CHAPTER 4

Proposed System

The proposed smart energy meter aims to overcome the limitations of traditional electromechanical, digital, and existing smart meters by providing real-time monitoring, predictive bill analysis, and load control using IoT-based automation. The system is designed to offer detailed energy insights by measuring both total energy consumption and individual load usage, enabling consumers to make informed decisions about their electricity consumption and reduce unnecessary energy wastage.

1. System Overview

The proposed system utilizes the ESP32 microcontroller, which acts as the central processing unit, interfacing with various sensors and modules to collect, process, and transmit energy-related data. The PZEM-004T energy meter is employed to measure voltage, current, power, and total energy consumption, while ACS712 current sensors are used to track the energy usage of individual loads. A 4-channel relay module enables users to remotely control appliances, allowing efficient energy management. Additionally, a DHT11 sensor is included to monitor temperature and humidity, helping to analyze environmental factors that might impact power consumption.

A 16x2 I2C LCD display is integrated to provide real-time energy readings, making it easy for users to check their power consumption at a glance. The system is designed to be cost-effective, efficient, and scalable, making it suitable for both residential and industrial applications.

2. Real-Time Monitoring and Data Processing

One of the key advantages of the proposed system is its real-time monitoring capability. Unlike conventional meters that require manual readings or offer delayed billing information, this system continuously measures energy usage and updates the user interface with real-time data. The ESP32 microcontroller

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

processes voltage and current values from the sensors and calculates power consumption using the formula:

$$\text{Power (W)} = \text{Voltage (V)} \times \text{Current (A)}$$

The energy consumption is then determined using:

$$\text{Energy (kWh)} = \text{Power (W)} \times \text{Time (h)} \times 3600 / 1000$$

This ensures accurate tracking of both total and individual load consumption, helping users identify high-energy-consuming appliances and take corrective actions to optimize electricity usage.

3. Predictive Bill Analysis

A major improvement in the proposed system is the implementation of predictive bill analysis. The system continuously monitors energy usage trends and calculates an estimated monthly electricity bill based on real-time consumption patterns. This feature allows users to anticipate their energy costs and make adjustments before the end of the billing cycle, preventing unexpected high charges.

The bill estimation is computed based on:

$$\begin{aligned} \text{Estimated Bill} &= \text{Total Energy Consumed} \times \text{Per-Unit Electricity Cost} \\ \{\text{Estimated Bill}\} &= \{\text{Total Energy Consumed}\} \setminus \{\text{Per-Unit Electricity Cost}\} \end{aligned}$$

This feature is particularly beneficial for households and businesses aiming to stay within a budgeted energy expenditure.

4. Load Control and Home Automation

To further enhance energy efficiency, the proposed system includes remote load control via a 4-channel relay module. Users can switch appliances ON/OFF directly through the system interface, providing greater control over electricity usage. This feature helps in:

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

Reducing unnecessary power wastage by turning off idle devices.

Enhancing safety by remotely disconnecting faulty or high-energy-consuming appliances.

Improving convenience by allowing automation of energy usage based on pre-set schedules.

5. User Interface and Display Mechanism

A 16x2 I2C LCD display is used to provide real-time power consumption details, including:

- Voltage (V) and Current (A)
- Total Power Consumption (W)
- Energy Consumed (kWh)
- Predicted Monthly Bill (Currency Unit)

This feature ensures that users can easily track their electricity usage without requiring external applications or cloud storage.

6. Advantages of the Proposed System

Compared to traditional and existing smart meters, the proposed system offers several advantages, including:

Real-time Energy Monitoring – Provides instant insights into power usage.
Predictive Bill Analysis – Estimates electricity costs before the billing cycle ends.

Individual Load Tracking – Monitors consumption of specific appliances.

Remote Load Control – Allows switching of appliances using relays.

Cost-Effective and Scalable – Can be expanded to include more sensors or integrate with smart home systems.

The proposed smart energy meter bridges the gap between conventional metering and modern IoT-based energy management solutions. By providing real-time monitoring, predictive billing, and remote load control, the system empowers

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

users to make energy-efficient decisions, reduce electricity costs, and contribute to sustainable energy consumption. This innovative approach ensures a smarter, more interactive, and cost-effective energy management system suitable for residential and industrial applications.

METHODOLOGY

This chapter deals with the design and development overview of the application, a brief description of hardware requirements, software system tools and its implementation.

The proposed system smart energy meter bridges the gap between conventional metering and modern IoT-based energy management solutions. By providing real-time monitoring, predictive billing, and remote load control, the system empowers users to make energy-efficient decisions, reduce electricity costs, and contribute to sustainable energy consumption. This innovative approach ensures a smarter, more interactive, and cost-effective energy management system suitable for residential and industrial applications.

Block Diagram

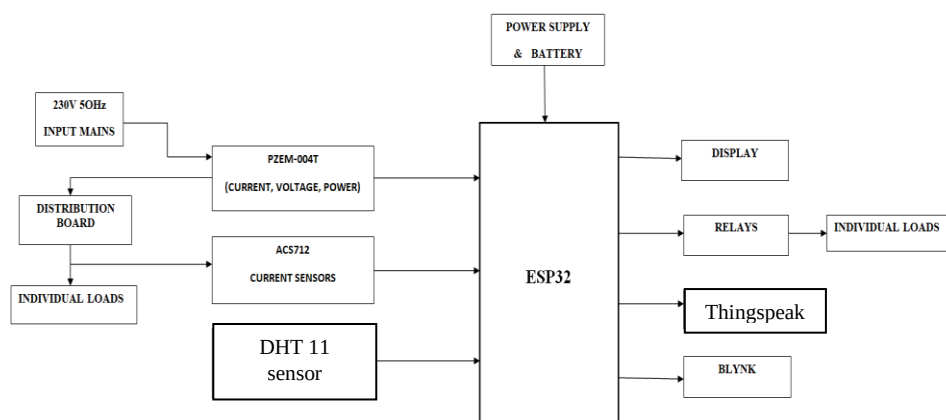


Fig 3.1: Block diagram of Smart Energy Meter

Working Procedure

The whole operation of this project is majorly divided into

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

7 steps.

Those are as mentioned below.

The smart energy meter operates through a systematic process that involves data acquisition, processing, display, and control. The system continuously monitors voltage, current, power, and energy consumption, providing real-time insights and enabling efficient energy management. The following steps outline the working procedure of the system:

1. Power Supply Initialization

- The system is powered using a regulated power supply, ensuring stable voltage to the ESP32 microcontroller and other components.
- The ESP32 initializes all the necessary peripherals, including the PZEM-004T energy meter, ACS712 current sensors, DHT11 temperature sensor, relays, and LCD display.

2. Data Acquisition

- Total Load Measurement:

The PZEM-004T sensor continuously measures voltage (V), current (A), power (W), energy consumption (kWh), power factor, and frequency from the main power source.

- Individual Load Monitoring:

The ACS712 sensors are connected to different appliances, allowing the system to track power consumption of individual loads separately.

- Environmental Data Collection:

The DHT11 sensor captures temperature and humidity data, which can help analyze the impact of environmental conditions on energy usage.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

3. Data Processing and Computation

- i. The ESP32 processes the raw data received from the PZEM-004T and ACS712 sensors.
- ii. Using the measured voltage and current, power consumption is calculated as:

$$\text{Power (W)} = \text{Voltage (V)} \times \text{Current (A)}$$

Energy consumption over time is computed as:

$$\text{Energy (kWh)} = \text{Power (W)} \times \text{Time (h)} \times 3600 / 1000$$

- iii. The system also estimates the monthly electricity bill based on the energy consumed and the per-unit cost of electricity, helping users predict their upcoming charges.

4. Displaying Real-Time Data

- i. The 16x2 I2C LCD display presents the real-time voltage, current, power consumption, and total energy usage.
- ii. Users can quickly check their power usage without needing additional software or applications.

5. Load Control Mechanism

- i. A 4-channel relay module is used to control individual electrical appliances.
- ii. Users can manually turn ON/OFF loads by pressing physical switches or using a predefined control mechanism.
- iii. This feature helps in optimizing energy consumption, preventing power wastage, and improving electrical safety.

6. Fault Detection and Safety Features

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

- i. The system continuously checks for high voltage conditions ($>250\text{V}$) and unusual power consumption patterns.
- ii. If an abnormal condition is detected, the user is alerted via the LCD display, and the system can trigger automatic load disconnection if necessary.
- iii. The relays can be programmed to disconnect loads during overvoltage or excessive current draw, preventing damage to appliances.

7. User Interaction and System Control

- i. The user can monitor real-time readings and control the appliances manually.
- ii. The predictive bill analysis feature allows users to estimate their energy expenses and take necessary actions to reduce costs.

The working procedure of the smart energy meter is structured to provide efficient, real-time energy monitoring and control. By integrating PZEM-004T for total load measurement, ACS712 for individual loads, and relays for automation, the system offers a complete energy management solution.

This automated energy monitoring system helps users make informed decisions, leading to energy savings, cost reduction, and improved electrical safety.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

Fig. Flow chart of proposed

4.5 DESIGN PROCEDURE

The hardware components of a smart energy meter play a crucial role in ensuring accurate energy measurement, efficient control, and reliable communication. This chapter provides a detailed analysis of the major hardware components used in the system, including their working principles, features, and interfacing with the ESP32 microcontroller. The components include energy measurement sensors, communication interfaces, display modules, and power supply circuits. By integrating these hardware elements, the smart energy meter provides real-time monitoring, predictive bill analysis, and efficient energy management

4.6 HARDWARE COMPONENTS

The smart energy meter system consists of various hardware components that work together to measure, display, and control energy consumption. These components include:

1. **ESP32 Microcontroller** – The core processing unit for data acquisition and communication.
2. **PZEM-004T Energy Meter** – Measures voltage, current, power, and energy consumption.
3. **ACS712 Current Sensors (x3)** – Measure the individual load currents.
4. **4-Channel Relay Module** – Controls individual load switching.
5. **DHT11 Sensor** – Monitors temperature and humidity.
6. **16x2 I2C LCD Display** – Displays real-time readings and energy consumption.
7. **Power Supply Unit** – Provides stable voltage levels for different components.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

Each component is selected based on its accuracy, efficiency, and compatibility with the ESP32 microcontroller to ensure seamless integration into the system. The following sections describe these components in detail.

4.7 DESCRIPTION OF MAJOR COMPONENTS

4.7.1 ESP 32 MICROCONTROLLER

The ESP32 is a powerful microcontroller with built-in Wi-Fi and Bluetooth capabilities, making it ideal for IoT applications such as smart energy meters. It serves as the central processing unit, handling data collection, communication, and control tasks.

4.7.1.1 Features and Specifications

1. Single or Dual-Core 32-bit LX6 Microprocessor with clock frequency up to 240 MHz.
2. 520 KB of SRAM, 448 KB of ROM and 16 KB of RTC SRAM.
3. Supports 802.11 b/g/n Wi-Fi connectivity with speeds up to 150 Mbps.
4. Support for both Classic Bluetooth v4.2 and BLE specifications.
5. 34 Programmable GPIOs.
6. Up to 18 channels of 12-bit SAR ADC and 2 channels of 8-bit DAC
7. Serial Connectivity include 4 x SPI, 2 x I²C, 2 x I²S, 3 x UART.
8. Ethernet MAC for physical LAN Communication (requires external PHY).
9. 1 Host controller for SD/SDIO/MMC and 1 Slave controller for SDIO/SPI.
10. Motor PWM and up to 16-channels of LED PWM.
11. Secure Boot and Flash Encryption.
12. Cryptographic Hardware Acceleration for AES, Hash (SHA-2), RSA, ECC and RNG.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

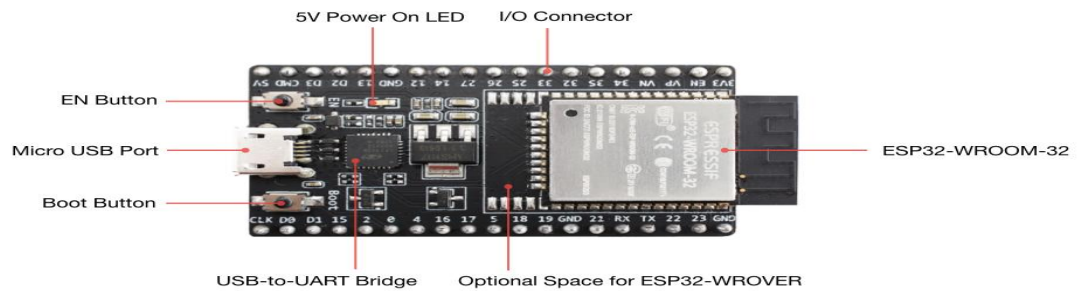


Fig. 4.1 ESP32

4.7.1.2 GPIO Pin Configuration

The ESP32 features multiple General-Purpose Input/output (GPIO) pins, which can be configured for different functionalities, including:

1. **Digital Input/ Output** – Used for controlling relays and reading sensor data.
2. **Analog Input** – Reads analog signals from sensors like ACS712.
3. **PWM Output** – Controls dimmable loads like LEDs.
4. **Interrupts** – Handles real-time responses for event-driven operations

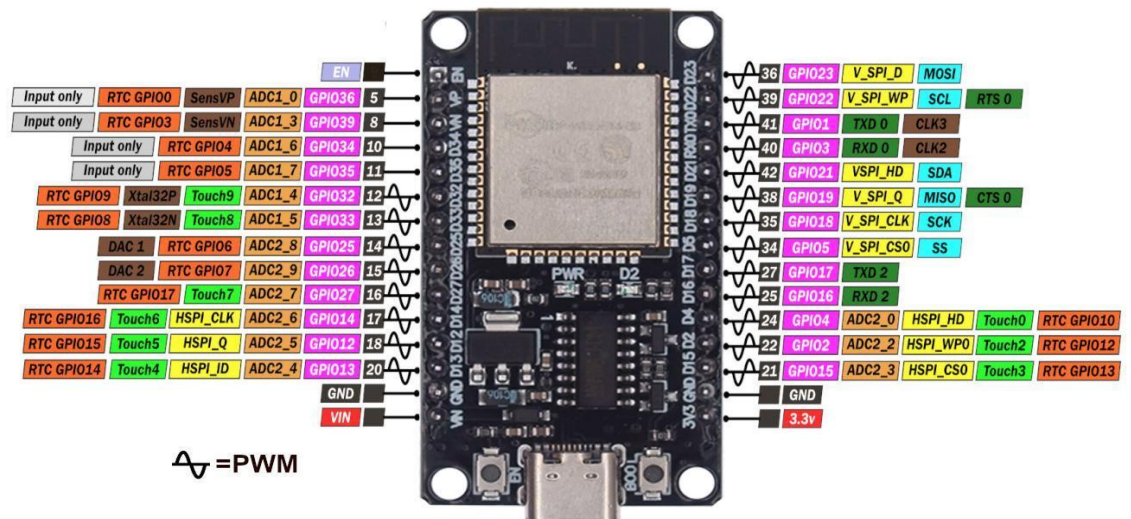


Fig. 4.2 ESP32 Pinout diagram

4.7.1.3 Communication Interfaces (I2C, UART, Wi-Fi)

The ESP32 supports multiple communication interfaces:

1. **I2C (Inter-Integrated Circuit)** – Used for interfacing with the LCD display and sensors.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

2. **UART (Universal Asynchronous Receiver-Transmitter)** – Communicates with the PZEM-004T energy meter.
3. **Wi-Fi** – Connects the system to a cloud-based platform for remote monitoring.
4. **Bluetooth** – Enables local wireless communication when needed.

4.7.1.4 Role in the Energy Meter System

The ESP32 microcontroller plays a crucial role in the smart energy meter system by:

1. **Collecting real-time energy data** from PZEM-004T and ACS712 sensors.
2. **Processing and analyzing power consumption** to generate insights.
3. **Controlling relays** to manage electrical loads efficiently.
4. **Displaying information** on the 16x2 LCD via I2C communication.
5. **Sending alerts and reports** through Wi-Fi connectivity to the cloud or a mobile application.

4.7.2 PZEM-004T Energy Meter



4.7.2.1 Working Principle

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

The PZEM-004T energy meter is a highly efficient module designed for measuring electrical parameters such as voltage, current, power, and energy consumption. It operates based on the principle of direct electrical measurement using an internal microcontroller and a current transformer. The module continuously monitors the electrical characteristics of an AC circuit and provides accurate readings via a digital communication interface.

4.7.2.2 Electrical Parameters

The key electrical parameters measured by the PZEM-004T energy meter include:

1. Voltage: 80–260V, 0.1V resolution, 0.5% accuracy
2. Current: 0–10A (10A model), 0–100A (100A model), 0.001A resolution, 0.5% accuracy
3. Active Power: 0–2.3kW (10A model), 0–23kW (100A model), 0.1W resolution, 0.5% accuracy
4. Power Factor: 0.00–1.00, 0.01 resolution, 1% accuracy
5. Frequency: 45–65Hz, 0.1Hz resolution, 0.5% accuracy
6. Active Energy: 0–9999.99kWh, 1Wh resolution, 0.5% accuracy, resettable via software
7. Over Power Alarm: Settable threshold, triggers alarm on exceedance
8. Communication: RS485 (UART to RS485), 9600 baud, 8N1

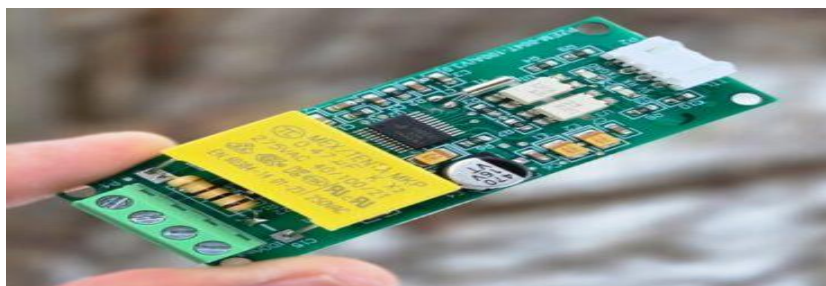


Fig. 4.4 PZEM-004T Sensor

4.7.2.3 Connection with ESP32

The PZEM-004T module interfaces with the ESP32 microcontroller to transmit energy data for further processing. The connection is established using:

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

A voltage input connection (L and N) to measure the line voltage.

A current transformer (CT) placed around the live wire to measure current flow.

UART communication lines (TX and RX) to send data to the ESP32



Fig. 4.5 PZEM-004T Connection Pinout

4.7.2.4 Communication Protocol (UART)

The PZEM-004T module communicates with the ESP32 using the **UART (Universal Asynchronous Receiver-Transmitter)** protocol. The UART interface allows:

1. **Asynchronous Serial Communication** – Ensures reliable data transmission without requiring clock synchronization.
2. **Full-Duplex Data Exchange** – Enables simultaneous sending and receiving of energy data.
3. **Baud Rate Configuration** – Typically set at **9600 bps** for stable data transmission.

Data from the PZEM-004T is read using standard serial communication commands, allowing the ESP32 to process and display energy consumption details in real time.

4.7.3 ACS712 Current Sensor

4.7.3.1 Overview of ACS712

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

The ACS712 is a Hall-effect-based current sensor that provides accurate and linear measurements of AC and DC currents. It operates by detecting the magnetic field generated by current flow and converting it into a proportional voltage output.

FEATURES AND BENEFITS

1. Low-noise analog signal path
2. Device bandwidth is set via the new FILTER pin
3. 5 μ s output rise time in response to step input current
4. 80 kHz bandwidth
5. Total output error 1.5% at TA= 25°C
6. Small footprint, low-profile SOIC-8 package
7. 1.2 m Ω internal conductor resistance
8. 2.4 kVRMS minimum isolation voltage from pins 1-4 to pins 5-8
9. 5.0 V, single supply operation
10. 66 to 185 mV/A output sensitivity
11. Output voltage proportional to AC or DC currents
12. Factory-trimmed for accuracy
13. Extremely stable output offset voltage
14. Nearly zero magnetic hysteresis
15. Ratio metric output from supply voltage

4.7.3.2 Measuring Individual Loads

Each ACS712 sensor is used to measure the current consumption of individual electrical loads. By placing the sensor in series with the load, it captures the real-time current flow, which is then processed by the ESP32.

4.7.3.3 Sensitivity and Output Voltage Relation

The ACS712 is available in different current ratings (5A, 20A, 30A), each with a different sensitivity:

1. **5A version:** 185 mV/A
2. **20A version:** 100 mV/A
3. **30A version:** 66 mV/A

The output voltage is centred around 2.5V at zero current and varies proportionally based on the measured current.



Fig. 4.7 Different ampere rating current sensors

4.7.3.4 Interfacing with ESP32

The ACS712 output is connected to an analog input pin of the ESP32. The ESP32 reads the sensor's voltage output, converts it into a current value, and uses it for further energy calculations.

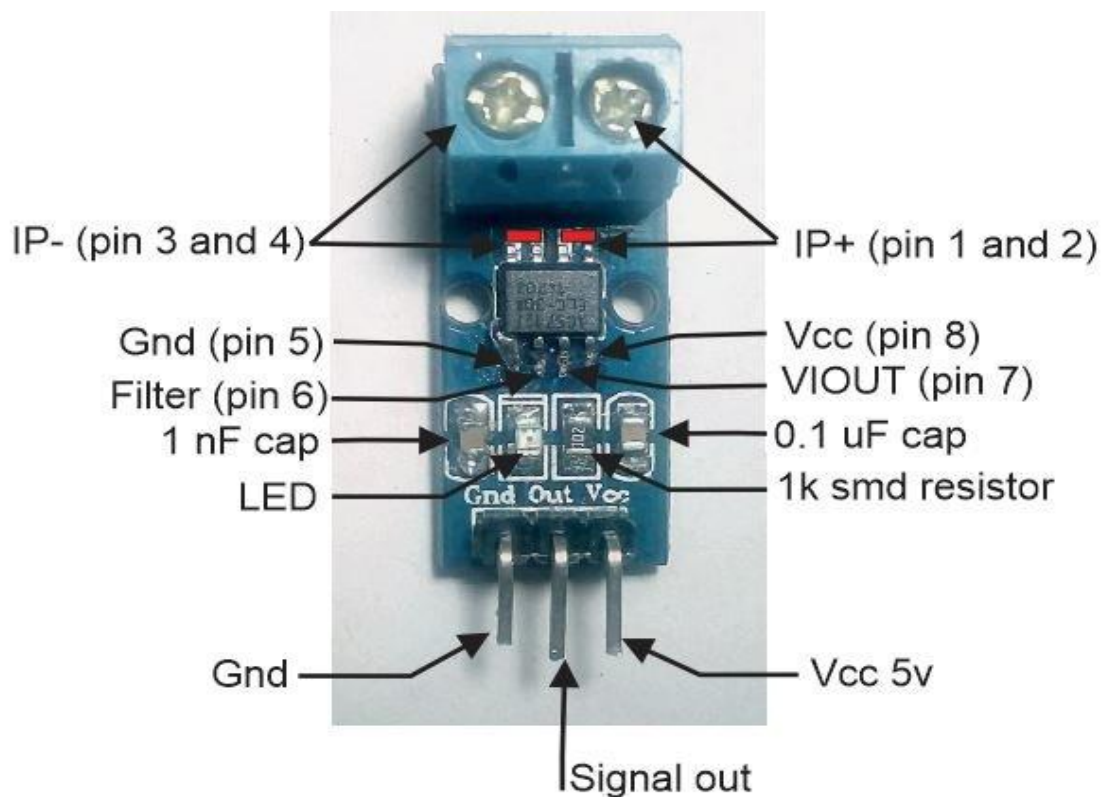


Fig. 4.8 Pinout and interface of ACS-712 current sensor

4.7.3.5 Calculation of Current and Power

The measured current is determined using:

Power consumption is calculated as:

where V is the voltage measured by the PZEM-004T module and I is the current from ACS712.

4.7.4 4-Channel Relay Module

The **4-channel relay module** is a crucial component in the **Smart Energy Meter** project, enabling the control of individual electrical loads remotely. This module operates as an electrically operated switch, allowing the ESP32 microcontroller to turn appliances on or off based on predefined conditions. The integration of relays in the system enhances automation, energy efficiency, and safety by preventing excessive energy consumption.

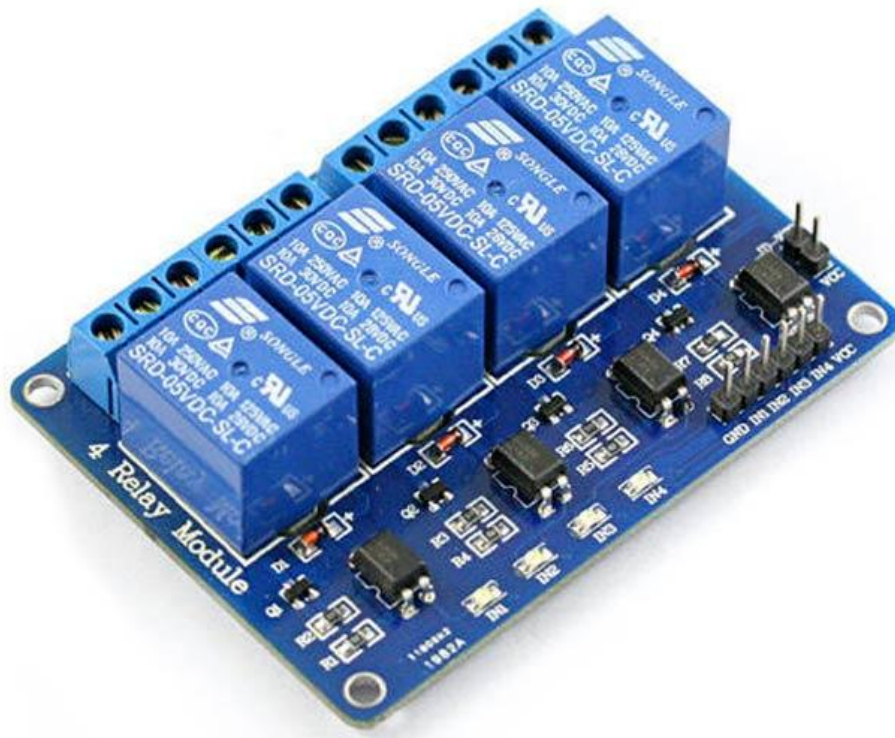


Fig. 4.9 4-channel relay module

4.7.4.1 Relay Module Working Principle

A relay is an electromechanical switch that uses an electromagnetic coil to control a mechanical switch. When a small input voltage is applied to the relay coil, it generates a magnetic field that pulls a mechanical lever, changing the state of the switch. The 4-channel relay module consists of four independent relays, each capable of handling high-voltage loads. It provides electrical isolation between the low-power control

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

circuit (ESP32) and high-power appliances, ensuring safe operation.

Each relay has three terminals:

1. **Common (COM):** The main terminal that connects to either Normally Open (NO) or Normally Closed (NC).
2. **Normally Open (NO):** When the relay is inactive, there is no connection. When activated, it connects to COM, allowing current to flow.
3. **Normally Closed (NC):** When the relay is inactive, this terminal is connected to COM. When activated, it disconnects.

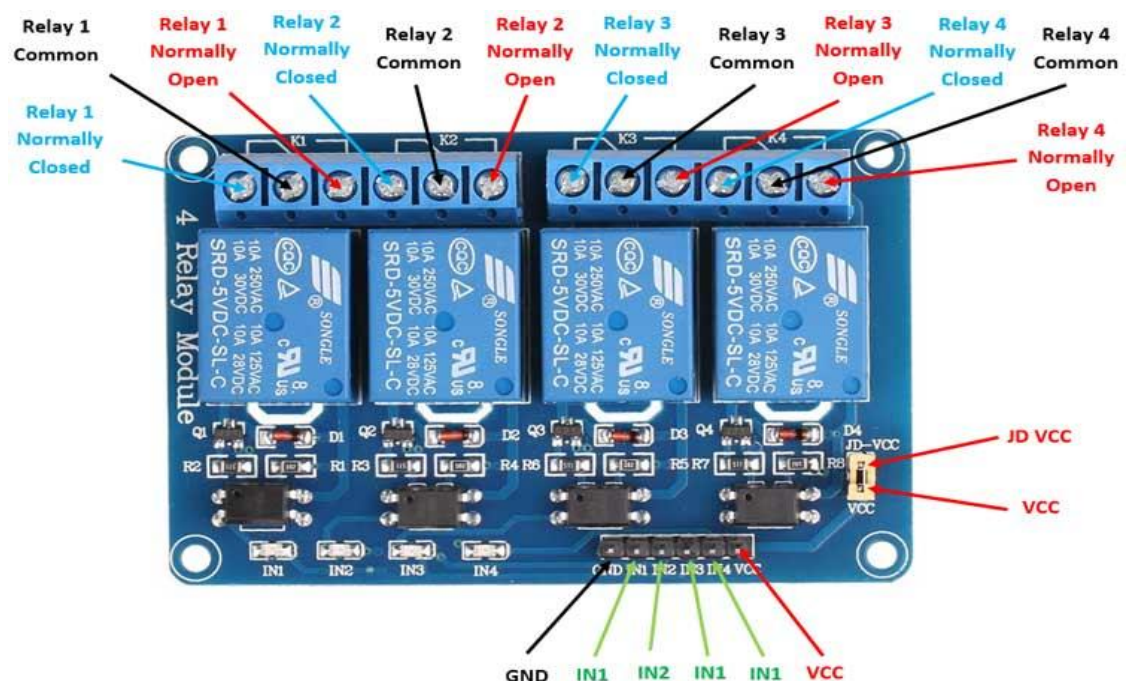


Fig. 4.10 Pinout of relay Module

In this project, the **NO** configuration is used, meaning that the appliances remain off by default and turn on only when the relay is triggered.

As we may know, relay is a device which is used to provide connection between two or more points or devices in response to the input signal applied. In other words, relays provide isolation between the controller and the device as devices may work on AC as well as on DC. However, they receive signals from a micro-controller which works on DC hence requiring a relay to bridge the gap. Relay is extremely useful when you need to control a large amount of current or voltage with small electrical signal. There are 5

parts in every relay

Electromagnet - It consists of an iron core wound by coil of wires. When electricity is passed through, it becomes magnetic. Therefore, it is called electromagnet. λ

Armature - The movable magnetic strip is known as armature. When current flows through them, the coil is energized thus producing a magnetic field which is used to make or break the normally open (N/O) or normally close(N/C)points. And the armature can be moved with direct current (DC) as well as alternating current (AC). λ

Spring - When no currents flow through the coil on the electromagnet, the spring pulls the armature away so the circuit cannot be completed. Set of electrical contacts - There are two contact points: Normally open - connected when the relay is activated, and disconnected when it is inactive.

Normally close - not connected when the relay is activated, and connected when it is inactive. λ Molded frame - Relays are covered with plastic for protection. The working principle of relay is simple. When power is supplied to the relay, currents start flowing through the control coil; as a result, the electromagnet starts energizing. Then the armature is attracted to the coil, pulling down the moving contact together thus connecting with the normally open contacts. So the circuit with the load is energized. Then breaking the circuit would a similar case, as the moving contact will be pulled up to the normally closed contacts under the force of the spring. In this way, the switching on and off of the relay can control the state of a load circuit.

4.7.4.2 Operating Voltages and Currents

The **4-channel relay module** operates at:

1. **Control voltage:** 5V (provided by the ESP32 using a level shifter, as ESP32 operates at 3.3V)
2. **Operating current:** Approximately 70mA per relay
3. **Load capacity:** 250V AC at 10A or 30V DC at 10A per relay
4. **Trigger voltage:** Low-level trigger (0V activates the relay)

Since the ESP32 operates at 3.3V logic and the relay module requires 5V signals, a transistor or level shifter is used to ensure proper switching.

4.7.4.3 Controlling of Individual Loads

Each relay in the **4-channel relay module** is assigned to control an individual load, such as bulbs and sockets. The relays are programmed to switch loads on or off based on various factors, including:

1. User commands via Blynk
2. Predefined energy consumption thresholds
3. Scheduled automation
4. Overload protection mechanisms

The ESP32 microcontroller processes the real-time energy data from the **PZEM-004T** and **ACS712 sensors**. If any predefined energy consumption limit is exceeded, the corresponding relay is turned off to prevent excessive usage. This automation helps in energy conservation and prevents electrical hazards.

4.7.4.4 Interfacing with ESP32

The **4-channel relay module** is interfaced with the ESP32 using GPIO pins. The connections are as follows:

VCC (Relay Module) → 5V (External Power Source)

GND (Relay Module) → GND (ESP32)

IN1, IN2, IN3, IN4 (Relay Module) → ESP32 GPIO Pins

Since the ESP32 operates at 3.3V and the relay module requires 5V, a level shifter or transistor-based switching circuit is used to ensure proper activation. The relay control logic in the ESP32 is programmed as follows:

Digital LOW (0V) → Relay ON (Appliance ON)

Digital HIGH (3.3V) → Relay OFF (Appliance OFF)

4.3.5 DHT11 Temperature Sensor

The DHT11 is a digital temperature and humidity sensor used in the Smart Energy

Meter project to monitor environmental conditions. It provides real-time temperature and humidity readings, which can be utilized for optimizing energy consumption and enhancing safety. The sensor is known for its low power consumption, cost-effectiveness, and ease of interfacing with microcontrollers like the ESP32.

4.7.5.1 Features and Specifications

The DHT11 sensor integrates a resistive humidity sensor and an NTC (Negative Temperature Coefficient) temperature sensor to measure atmospheric conditions accurately. The key features and specifications of the DHT11 are:

1. Operating Voltage: 3.3V to 5V
2. Temperature Range: 0°C to 50°C ($\pm 2^\circ\text{C}$ accuracy)
3. Humidity Range: 20% to 90% RH ($\pm 5\%$ accuracy)
4. Sampling Rate: 1 reading per second (1Hz)
5. Response time : 6s-15s
6. Data Interface: Single-wire digital signal
7. Power Consumption: 2.5mA max during data transmission

The DHT11 operates with a one-wire communication protocol, requiring only a single data line for communication with the ESP32, making it easy to interface with minimal wiring.

4.7.5.2 Temperature and Humidity Sensing

The DHT11 sensor measures:

1. Temperature using a thermistor that changes resistance based on temperature variations.
2. Humidity using a capacitive humidity sensor, where the capacitance varies with the moisture level in the surrounding air.

The sensor collects data and transmits it as a digital signal to the ESP32, ensuring precise readings with minimal noise.

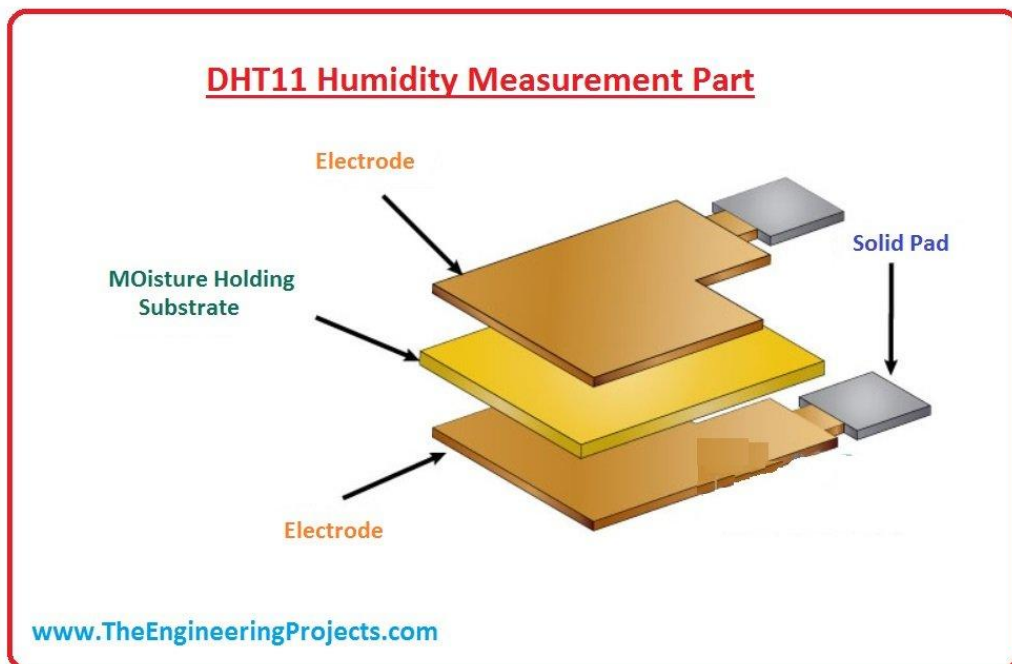
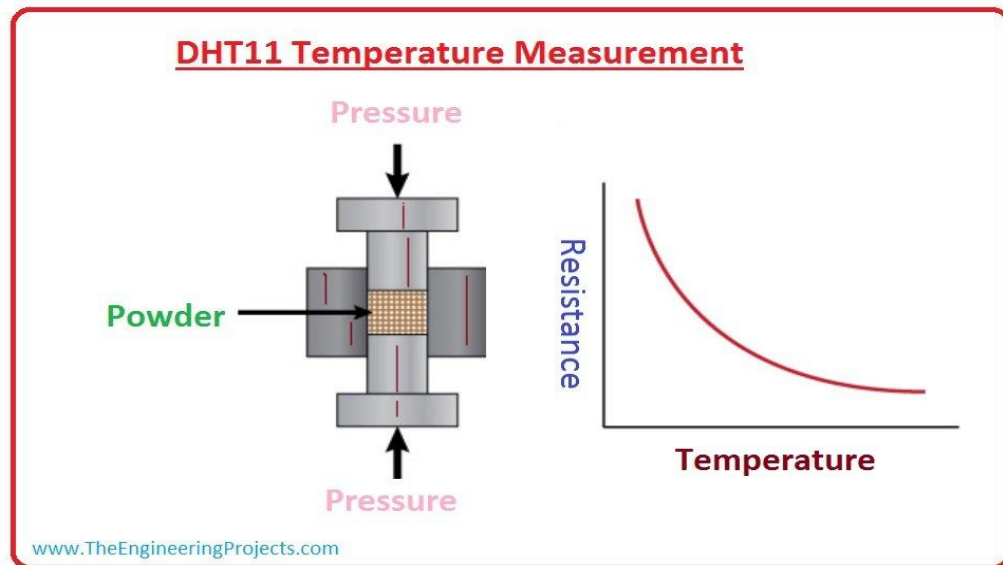


Fig. 4.11 DHT11 Sensor working

Use in Smart Energy Meter

In this project, the DHT11 sensor enhances energy efficiency by:

1. Monitoring ambient temperature to optimize appliance usage.
2. Providing real-time humidity data, which can be displayed on the LCD and monitored remotely.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

3. Triggering alerts for extreme temperature conditions, helping to prevent overheating or electrical hazards.

4.7.5.3 Connection and Data from ESP32

- DHT11 Pinout consists of 4 Pins in total, listed below from left to right:
 1. **VCC**: Need to provide +5V at this pinout.
 2. **Data**: It's the digital output pin, that gives either 0V or 5V.
 3. **NC**: Not Connected. (It's left open for future design)
 4. **GND**: Need to provide Ground at this pinout.

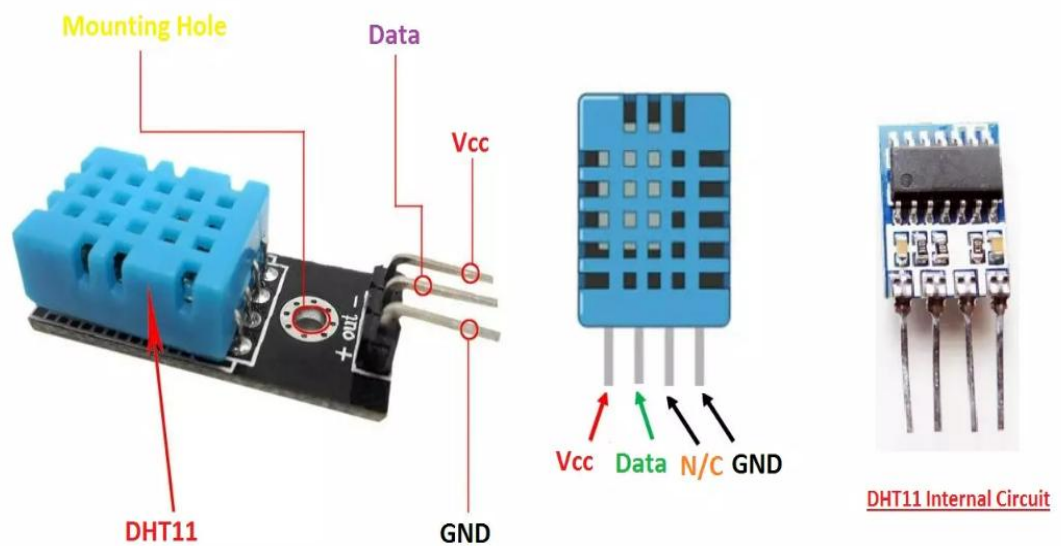


Fig.4.12 DHT11 Pinout

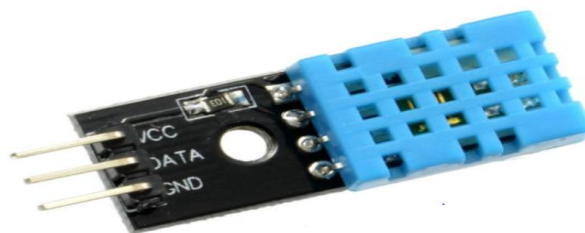


Fig.4.13 DHT 11 Sensor

4.7.6 16x2 I2C LCD Display

The **16x2 I2C LCD Display** is used in the **Smart Energy Meter** project to display real-time energy consumption data, temperature, humidity, and system alerts. The **I2C** (Inter-Integrated Circuit) communication interface allows easy integration with the **ESP32** using only two data lines, reducing wiring complexity compared to traditional parallel LCD displays.

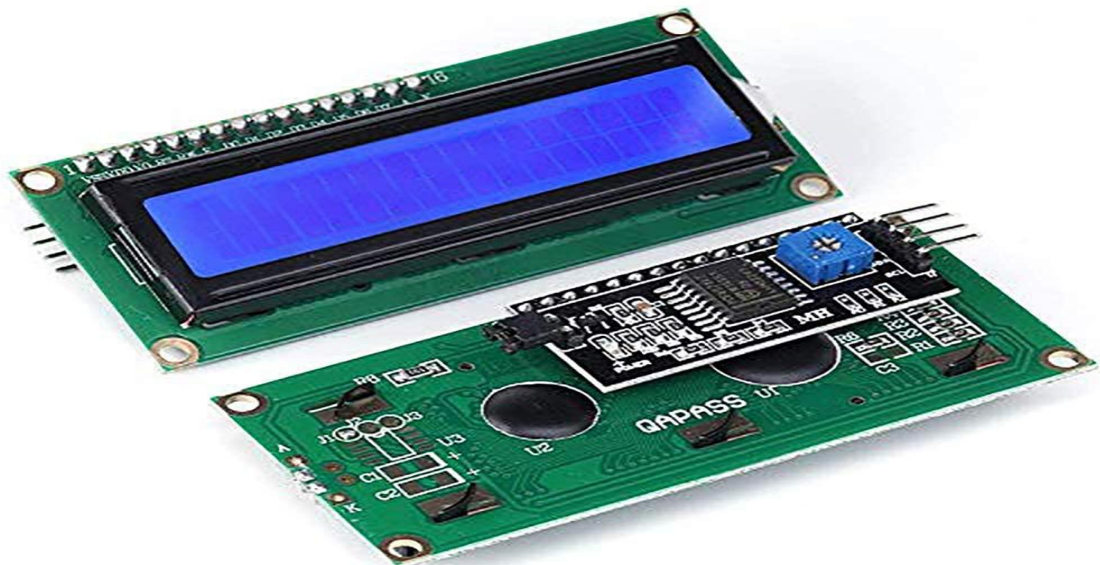


Fig.4.14 16x2 LCD Display with I2C

4.7.6.1 LCD Display Specifications

The **16x2 LCD** is a character display module that can show **two lines of 16 characters each**. With the **I2C module (PCF8574 IC)**, the display operates efficiently with minimal GPIO usage. The key specifications are:

- **Display Type:** 16x2 character LCD
- **Controller IC:** HD44780
- **Interface:** I2C (via PCF8574 IC)

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

- **Operating Voltage:** 5V
- **Backlight:** LED (usually blue with white text)
- **Contrast Adjustment:** Potentiometer on the I2C adapter

4.7.6.2 I2C Communication Protocol

I2C (Inter-Integrated Circuit) is a **two-wire serial communication protocol** that allows multiple devices to communicate over a shared data bus. The **ESP32** communicates with the **16x2 I2C LCD** using the following two lines:

- **SDA (Serial Data Line)** → Transmits data
- **SCL (Serial Clock Line)** → Synchronizes communication



Fig. 4.15 LCD Pinout

I2C Address

Each I2C device has a unique **7-bit address**. The **16x2 I2C LCD module** usually operates at one of the following addresses:

- **0x27 (Default)**
- **0x3F (Alternative)**

If multiple I2C devices are connected, an **I2C scanner code** can be used to detect the exact address.

4.7.6.3 Interfacing with ESP32

The **16x2 I2C LCD** is connected to the **ESP32** using only **four wires**:

LCD Pin	Function	ESP32 Connection
VCC	Power (5V)	5V (ESP32)
GND	Ground	GND (ESP32)
SDA	Data Line	GPIO21 (ESP32 SDA)
SCL	Clock Line	GPIO22 (ESP32 SCL)

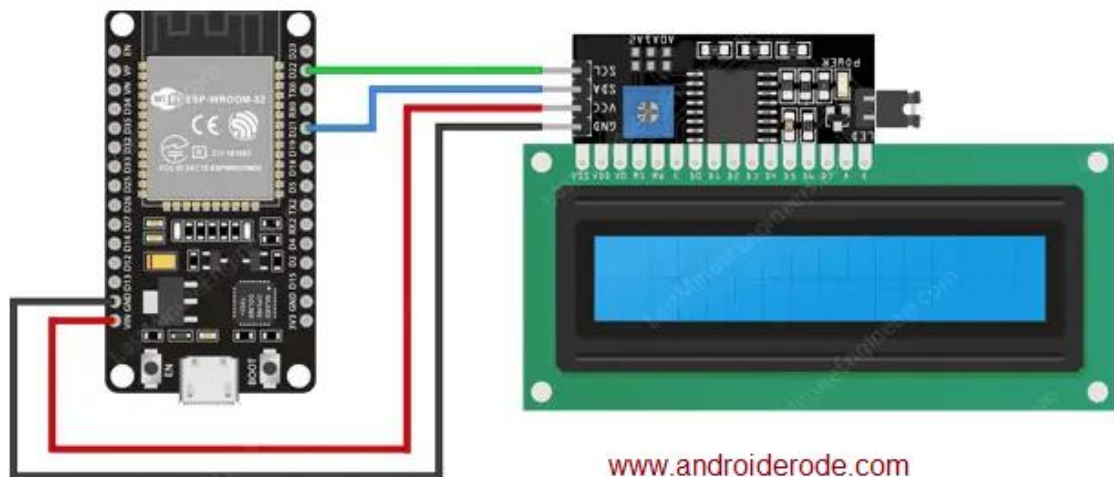


Fig. 4.16 LCD Interfacing with ESP32

4.7.7 Power Supply

The **power supply unit** in the **Smart Energy Meter** project is responsible for providing stable **5V and 3.3V** power to the ESP32, sensors, relays, and display

modules. The system uses a **regulated power supply circuit** to ensure smooth operation and prevent voltage fluctuations.

4.7.7.1 Power Requirements for ESP32

The **ESP32** is the main microcontroller in the project and operates at **3.3V**. It requires a stable power supply to function properly, especially during Wi-Fi communication, which can cause voltage spikes.

- **Operating Voltage:** 3.3V
- **Current Consumption:**
 - **Idle Mode:** ~80mA
 - **Wi-Fi Active:** 250mA - 500mA
 - **Peak Consumption:** ~1A (during transmission bursts)
- **Power Input Options:**
 - **Micro-USB (5V input)** → Uses the onboard **AMS1117-3.3V regulator**
 - **VIN Pin (5V input)** → Direct **5V supply** from the external regulator
 - **3.3V Pin** → Direct **3.3V regulated supply**

To ensure stability, a **capacitor (10 μ F or higher)** is recommended across the ESP32's power pins.

4.7.7.2 5V and 3.3V Voltage Regulation

Since different components in the project require either **5V or 3.3V**, a proper voltage regulation system is implemented.

- **5V Supply:**
 - Used for **Relay Module, LCD Display, and Sensors**
 - Generated using a **5V adapter (2A recommended)**
- **3.3V Supply:**
 - Used for the **ESP32, ACS712 Current Sensors, and DHT11 Temperature Sensor**

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

- Regulated using an **AMS1117-3.3V voltage regulator**

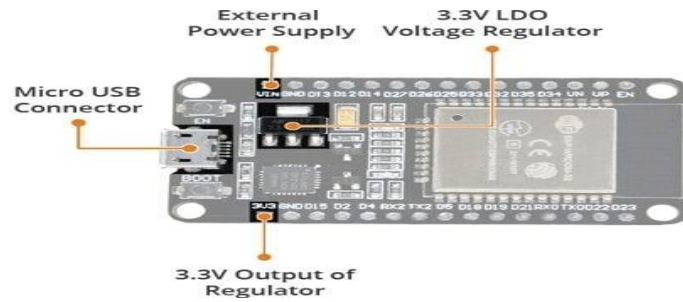


Fig. 4.17 ESP32 Power Supply

Voltage Regulators Used

1. **AMS1117-3.3V Regulator** (For ESP32)
 - **Input Voltage:** 5V
 - **Output Voltage:** 3.3V
 - **Max Current Output:** 800mA
 - **Capacitors Used:** 10 μ F (input) & 10 μ F (output)
2. **5V Power Supply Module** (For Relays & Display)
 - Converts **AC mains (230V) to stable 5V DC**
 - Provides enough current for relays and LCD

This dual voltage regulation ensures the safe operation of all components without overloading the ESP32.

4.7.7.3 Power Supply Circuit Design

Based on the circuit diagram, the **power supply circuit** consists of a **5V adapter**, **voltage regulators**, and **necessary filtering capacitors**.

Power Supply Connections in the Circuit:

- 5V Adapter → Powers the Relay Module, LCD Display, and ACS712 Sensors
- AMS1117-3.3V Regulator → Converts 5V to 3.3V for ESP32 and DHT11
- Filtering Capacitors → Reduce voltage fluctuations and noise
- Schottky Diode (1N4007) → Prevents reverse polarity damage

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

Component	Voltage Requirement	Power Source
ESP32	3.3V	AMS1117-3.3V
Relay Module	5V	5V Adapter
LCD Display	5V	5V Adapter
ACS712 Sensors	5V	5V Adapter
DHT11 Sensor	3.3V	ESP32 (3.3V Pin)

Power Supply Circuit Overview

- The **5V adapter** provides power to **relays, LCD, and sensors**.
- The **AMS1117-3.3V regulator** steps **5V down to 3.3V** for the ESP32.
- **Capacitors** (10 μ F and 100 μ F) stabilize the voltage to prevent fluctuations.
- The **ESP32 can be powered via its 5V pin (from the external adapter) or 3.3V pin (from AMS1117).**

This **regulated power supply design** ensures that all components receive the required voltage levels safely, avoiding power surges and instability.

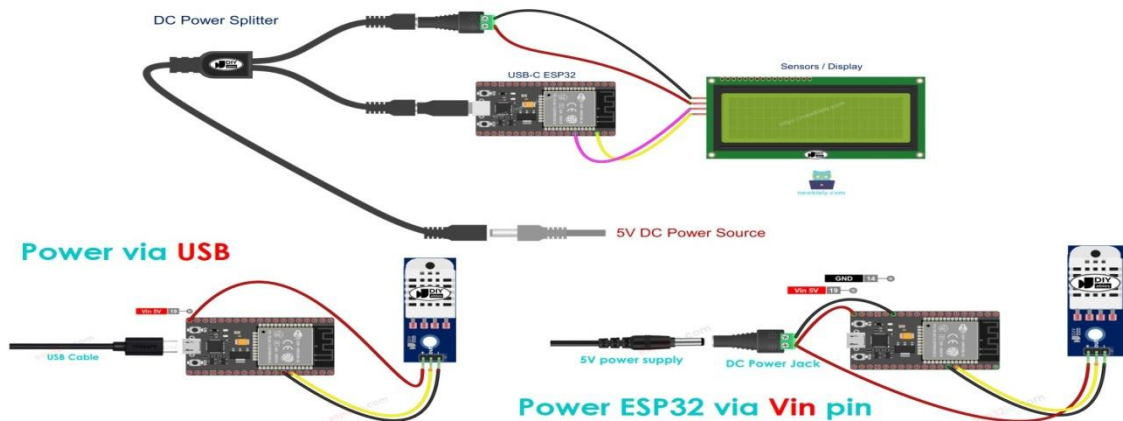


Fig. 4.18 Different types of power supply to ESP32

4.8 SOFTWARE TOOLS

ESP32 with Arduino IDE



To program your boards, you need an IDE to write your code. For beginners, we recommend using Arduino IDE. While it's not the best IDE, it works well and is simple and intuitive to use for beginners. After getting familiar with Arduino IDE and you start creating more complex projects, you may find it useful to use [VS Code with the Platformio extension](#) instead.

If you're just getting started with the ESP32, start with [Arduino IDE](#).

Installing Arduino IDE

To run Arduino IDE, you need to install JAVA on your computer. If you don't, go to the following website to download and install the latest version: <http://java.com/download>.

Downloading and Installing Arduino IDE

To download the Arduino IDE, visit the following URL:

- <https://www.arduino.cc/en/Main/Software>

Go to the [Arduino website and download the version](#) for your operating system.

Arduino IDE 2.3.2

The new major release of the Arduino IDE is faster and even more powerful! In addition to a more modern editor and a more responsive interface it features autocompletion, code navigation, and even a live debugger.

For more details, please refer to the [Arduino IDE 2.0 documentation](#).

Nightly builds with the latest bugfixes are available through the section below.

SOURCE CODE

The Arduino IDE 2.0 is open source and its source code is hosted on [GitHub](#).

DOWNLOAD OPTIONS

Windows Win 10 and newer, 64 bits
Windows MSI installer
Windows ZIP file

Linux ApplImage 64 bits (X86-64)
Linux ZIP file 64 bits (X86-64)

macOS Intel, 10.15: "Catalina" or newer, 64 bits
macOS Apple Silicon, 11: "Big Sur" or newer, 64 bits

[Release Notes](#)

Fig. 4.19 Arduino Ide 2.3.2 version download

- **Windows:** run the file downloaded and follow the instructions in the installation guide.
- **Mac OS X:** copy the downloaded file into your application folder.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

- **Linux:** extract the downloaded file, and open the `arduino-ide` file that will launch the IDE.

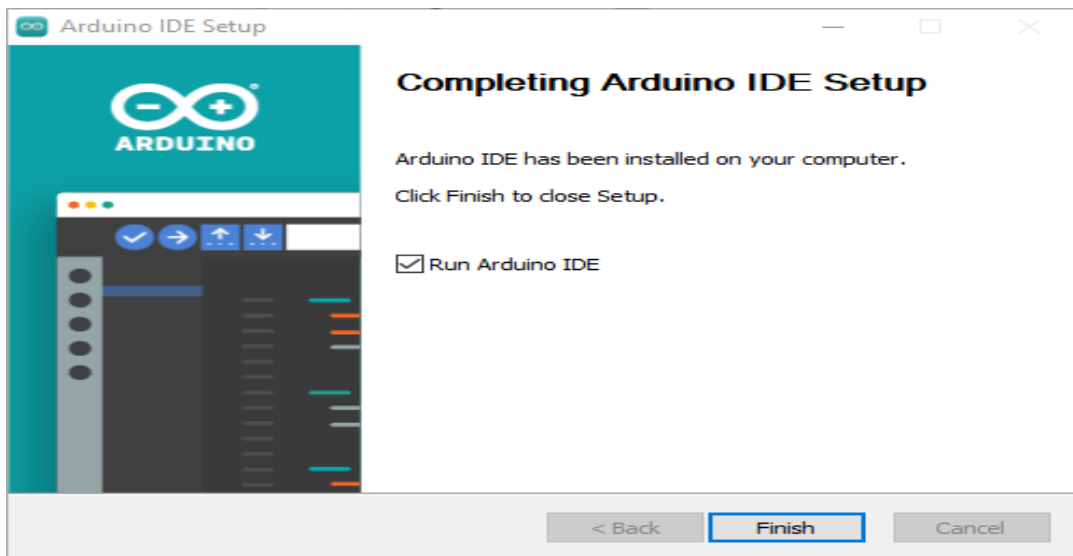


Fig. 4.20 Settingup the Arduino

If you have doubts, you can go to the [Arduino Installation Guide](#).

Do you need an ESP32 board? [You can buy it here](#).

Recommended reading: [ESP32 Development Boards Review and Comparison](#)

Installing the ESP32 in Arduino IDE

To install the ESP32 board in your Arduino IDE, follow these next instructions:

1. In your Arduino IDE 2, go to **File > Preferences**.

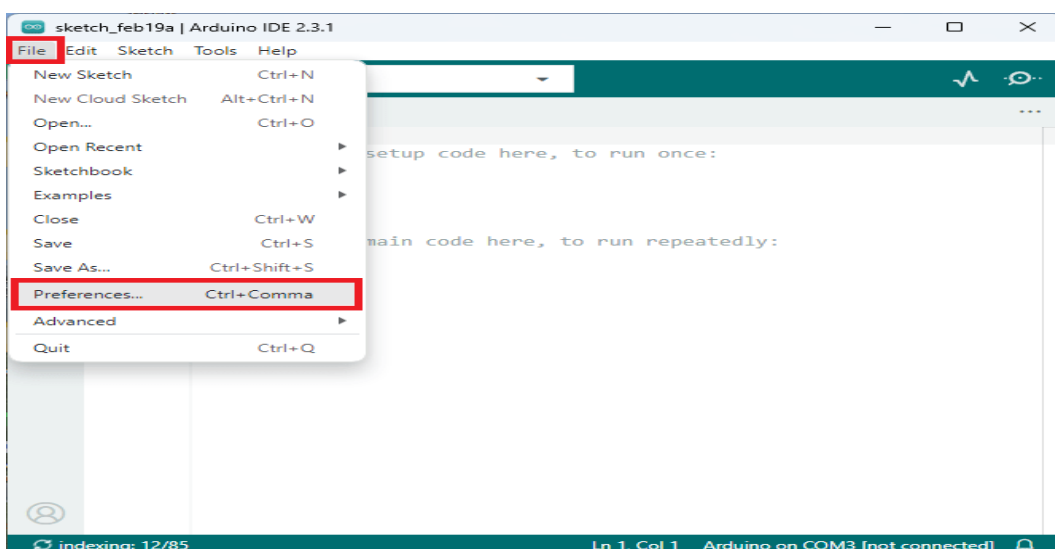
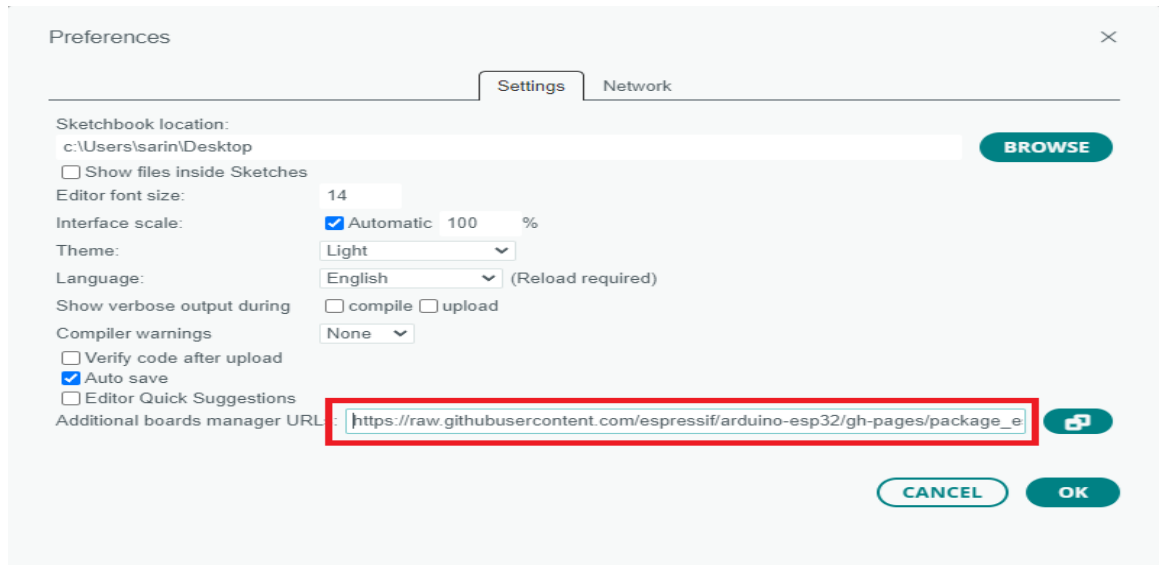


Fig. 4.21 Selecting the Preferences

2. Copy and paste the following line to the **Additional Boards Manager URLs** field.
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS



Note: if you already have the ESP8266 boards URL, you can separate the URLs with a comma, as follows:

http://arduino.esp8266.com/stable/package_esp8266com_index.json,
https://raw.githubusercontent.com/espressif/arduino-esp32/gh-pages/package_esp32_index.json

3. Open the Boards Manager. You can go to **Tools > Board > Boards Manager...** or you can simply click the **Boards Manager** icon in the left-side corner.



Fig. 4.22. opening Board manager

4. Search for **ESP32** and press the install button for **esp32 by Espressif Systems version 3.X**.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

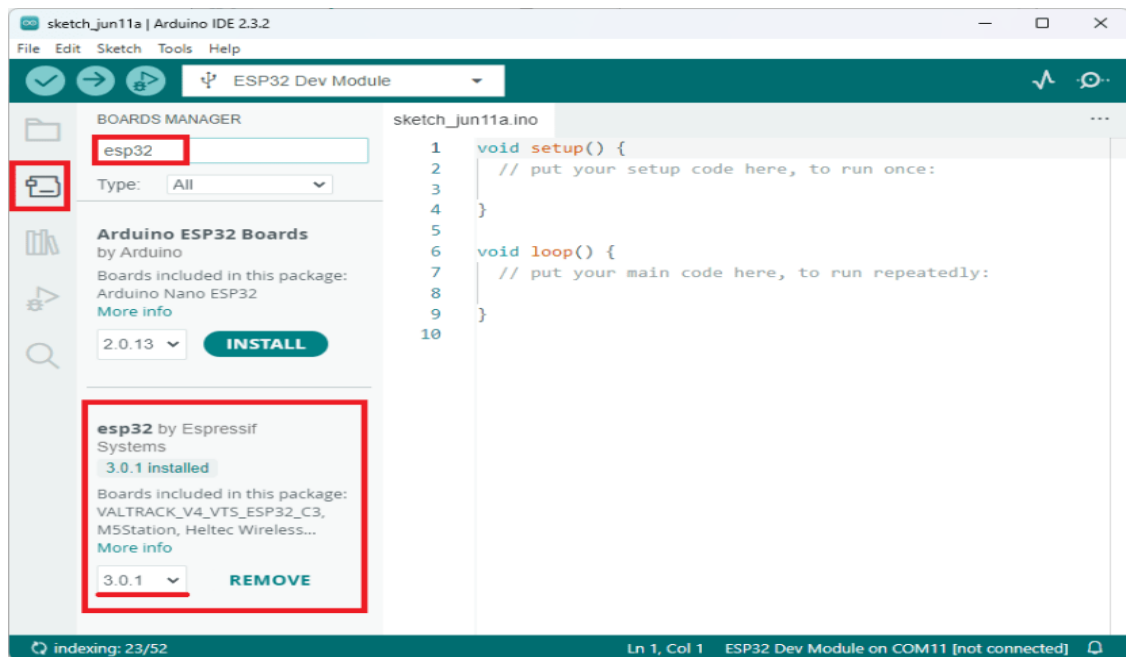


Fig.4.23 Select necessary boards in Board manager

That's it. It should be installed after a few seconds.

After this, restart your Arduino IDE.

Then, go to **Tools** > **Board** and check that you have ESP32 boards available.

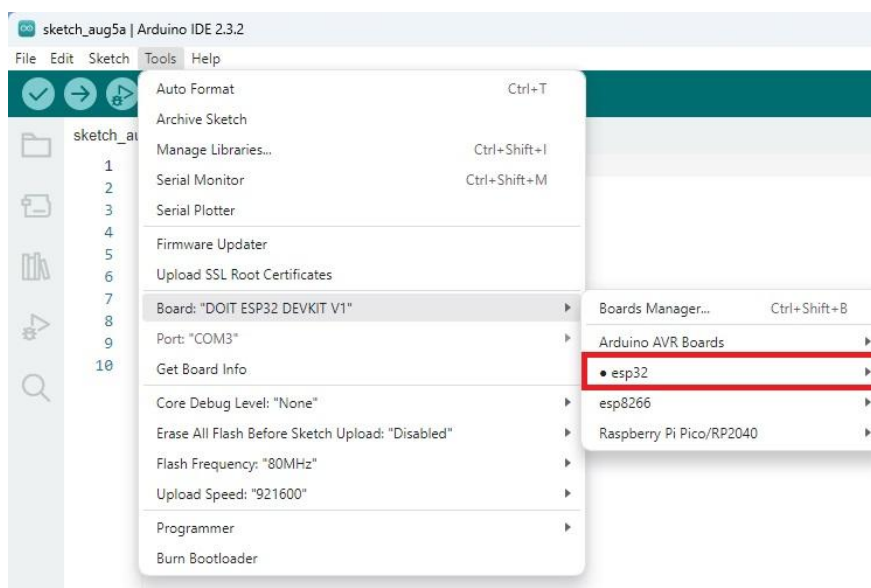


Fig. 4.24 Select the ESP32 board in tools

Now, you're ready to start programming your ESP32 using Arduino IDE.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

Testing the Installation and Uploading Code to the ESP32

Now, let's check if the installation was successful and if we can upload new code to the ESP32 board.

We'll simply upload an example sketch from the library of available examples.

Connect your ESP32 development board to your computer using a USB cable. If you have an ESP32 DEVKIT DOIT board, the built-in red LED will turn on. This indicates the board is receiving power.

Important: you must use a USB cable with data wires. Some USB cables from chargers or power banks are power only and they don't transfer data—these won't work.

With your Arduino IDE open, follow these steps:

1) Select your Board in **Tools** > **Board** menu or on the top drop-down menu, click on "Select other board and port..."

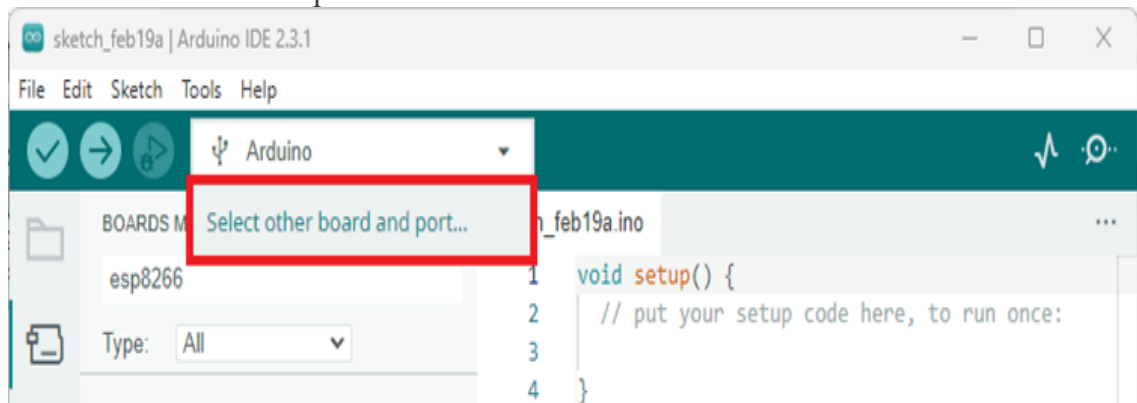


Fig.4.25 Select the Board and Port

A new window, as shown below, will open. Search for your ESP32 board model.

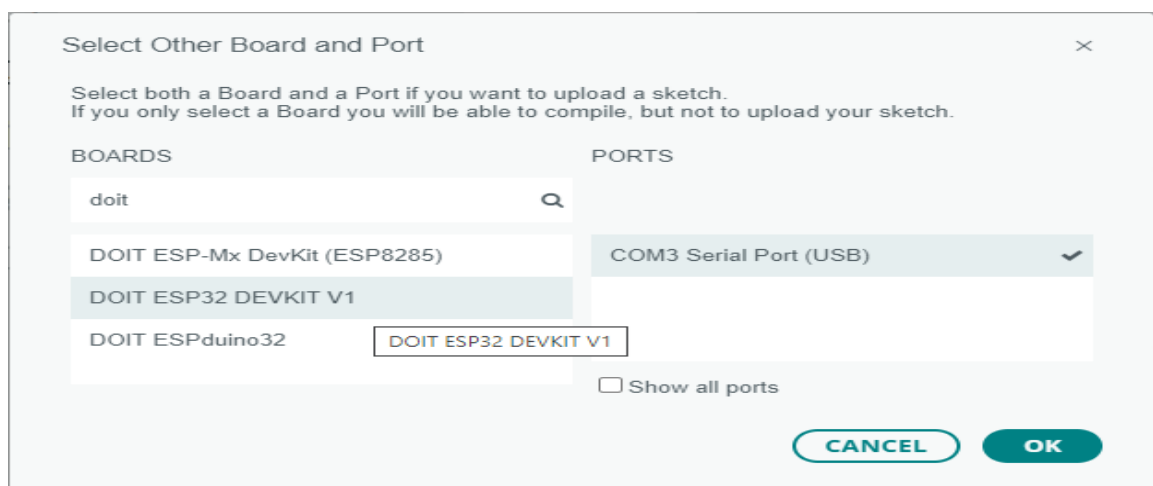


Fig.4.26 Selecting the ESP32 DEVKIT V1 in Board and Port

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

Select the board model you're using, and the COM port. In our example, we're using the DOIT ESP32 DEVKIT V1. Click **OK** when you're done.

Installing the USB Drivers

If you don't see the COM Port in your Arduino IDE, you probably need to install the USB-to-UART drivers on your computer. Most ESP32 boards use either the CP210x or the CH340 chip depending on the board you're using.

If you need to install the CP210x drivers, we recommend taking a look at this tutorial: [Install ESP32/ESP8266 USB Drivers – CP210x USB to UART Bridge](#)

2) Open the following example— it searches for wi-fi networks within the range of your board.

- ESP32: **File** > **Examples** > **WiFi (ESP32)** > **WiFiScan**

3) A new sketch opens in your Arduino IDE:

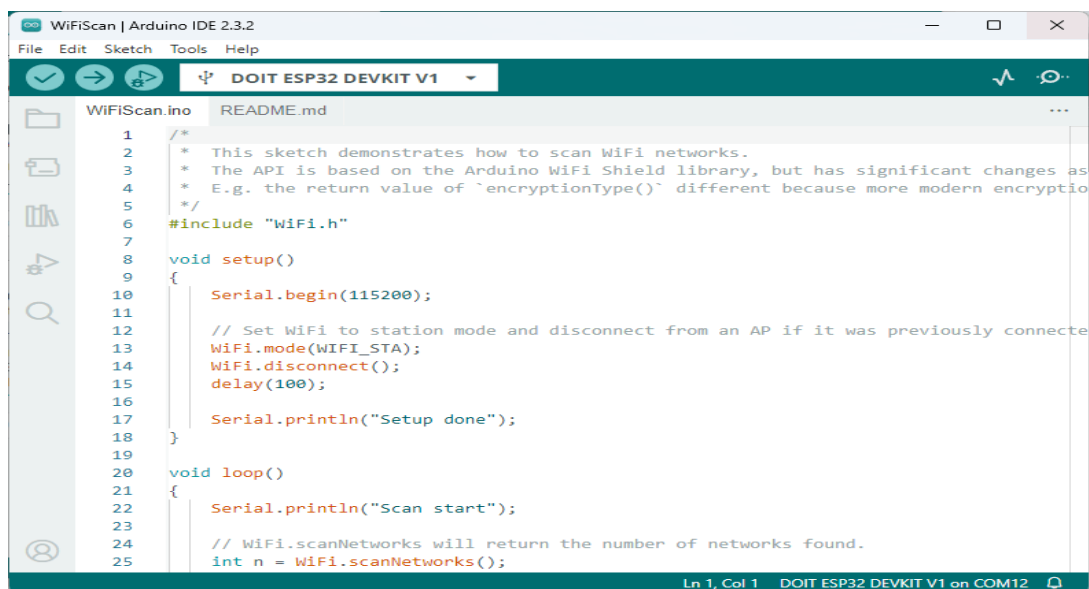


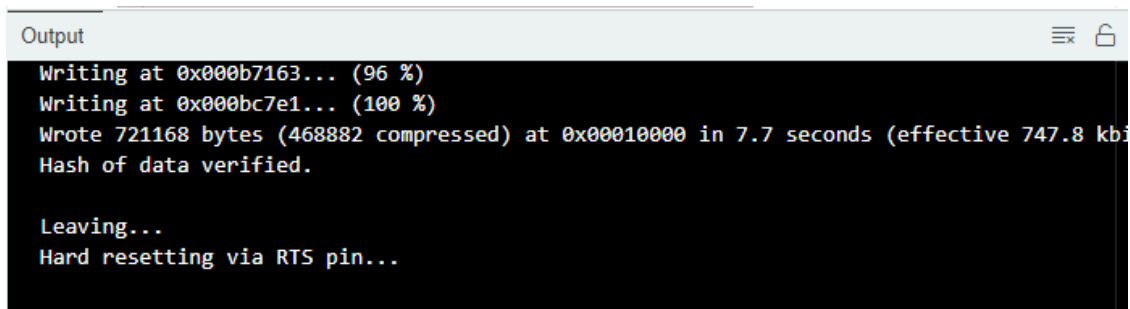
Fig. 4.27 Writing a code

4) Press the **Upload** button in the Arduino IDE. Wait a few seconds while the code compiles and uploads to your board.

Note: if you see a lot of dots on the debugging window, followed by an upload error, that means your board doesn't go into flashing mode automatically. Click the Upload button again, and when you start seeing the dots on the debugging window, press the onboard BOOT button for a couple of seconds.

5) If everything went as expected, it will upload successfully after a few seconds. You'll get a similar message:

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS



```
Output
Writing at 0x000b7163... (96 %)
Writing at 0x000bc7e1... (100 %)
Wrote 721168 bytes (468882 compressed) at 0x00010000 in 7.7 seconds (effective 747.8 kb/s)
Hash of data verified.

Leaving...
Hard resetting via RTS pin...
```

Fig.4.28 Successfully uploaded the code to ESP32

6) Open the Arduino IDE Serial Monitor at a baud rate of 115200:

7) Press the ESP32 on-board Enable/RESET button and you should see the networks available near your board.

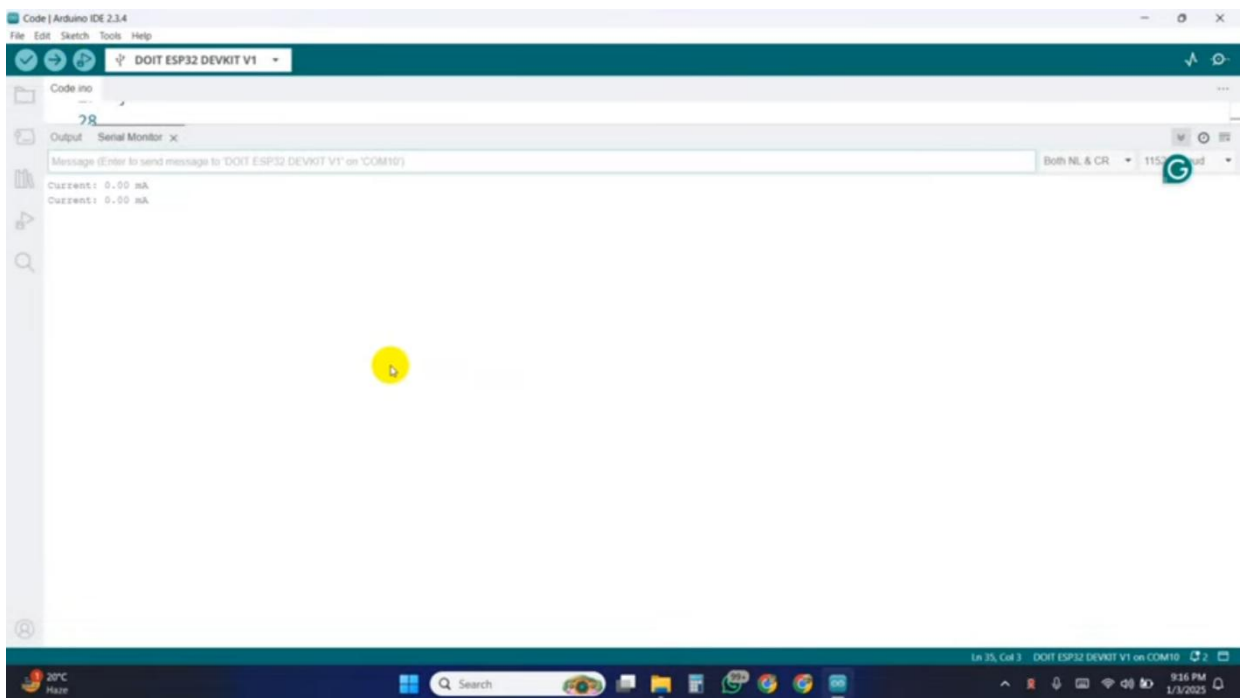


Fig. 4.29 Serial Monitor

If you're having issues uploading code to your ESP32 board, we recommend taking a quick look at the following troubleshooting guide: [ESP32 Troubleshooting Guide](#).

4.9 Blynk Application

Blynk Setup for Smart Energy Meter

Introduction

In this Smart Energy Meter project, the **Blynk IoT platform** is used to enable **remote monitoring** and **control** of the system through a smartphone. Blynk provides an intuitive and user-friendly interface that allows users to monitor **real-time voltage, current, power, energy consumption, and temperature** while also **controlling connected electrical loads** (relays) wirelessly.

Objective of Blynk Integration

The main objectives of integrating Blynk into the Smart Energy Meter are:

- To monitor energy parameters remotely and in real-time.
- To control electrical loads from a mobile device.
- To improve user convenience and energy efficiency.
- To provide a scalable and reliable IoT-based solution.

Blynk Setup Procedure

1. Installation of Blynk Application

- Download the **Blynk IoT App** from the **Google Play Store** (for Android) or the **Apple App Store** (for iOS devices).
- Install and open the application on the mobile device.
- **Sign up** for a new account using an email address or **log in** if an account already exists.

2. Creation of a New Blynk Project

- Tap on “**New Project**” in the Blynk app.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

- Set a **Project Name** such as *Smart Energy Meter*.
- Select **Device Type**: Choose **ESP32**.
- Set the **Connection Type**: Select **Wi-Fi**.
- Tap **Create**.
- A **unique Auth Token** will be generated and sent to the registered email. This Auth Token is essential for connecting the ESP32 to Blynk Cloud.

3. Designing the Blynk Mobile Dashboard

To create an effective monitoring and control dashboard, the following widgets must be added:

Widget	Purpose	Virtual Pin Assignment
Gauge	Display Voltage	V1
Gauge	Display Current	V2
Gauge	Display Power	V3
Gauge	Display Energy Consumption (kWh)	V4
Gauge	Display Temperature	V5
Button	Control Load 1 (Relay 1)	V10
Button	Control Load 2 (Relay 2)	V11
Button	Control Load 3 (Relay 3)	V12
Button	Control Load 4 (Relay 4)	V13

Widget Settings:

- **Gauges:** Configure min-max values based on expected sensor ranges (e.g., 0–250V for voltage, 0–30A for current).
- **Buttons:** Set to “Switch” mode for toggling loads ON or OFF.

Each widget is mapped to a **Virtual Pin** that links the mobile app controls to the ESP32 microcontroller actions.

4. Connecting ESP32 to Blynk Cloud

- The ESP32 microcontroller is programmed using the Arduino IDE.
- The **Auth Token**, **Wi-Fi SSID**, and **Wi-Fi Password** must be entered in the program code.
- Blynk library functions are used to send sensor data to virtual pins and read button commands to operate relays.

5. Uploading and Testing the System

- After uploading the firmware to ESP32, the board connects to the local Wi-Fi network and then to the Blynk Cloud.
- The Blynk mobile app dashboard can now display **real-time sensor values** and control **relay outputs**.
- Users can observe live voltage, current, power, energy consumption, and temperature readings.
- Loads can be switched ON or OFF remotely by tapping the buttons on the mobile screen.

Advantages of Blynk Integration

- **Real-Time Monitoring:** Immediate visibility of power usage and environmental conditions.
- **Remote Load Control:** Devices can be controlled from anywhere in the world using a smartphone.
- **User-Friendly Interface:** Simple and visually appealing dashboard setup without complex coding.
- **Scalable and Flexible:** Additional loads, sensors, or alerts can be added easily by modifying the app layout and code.
- **Enhanced Energy Management:** Enables users to track energy consumption and control devices, leading to better energy efficiency and cost savings.

Conclusion

The integration of the Blynk platform with the Smart Energy Meter system provides users with a **powerful and convenient tool** for managing energy usage. Through a combination of real-time data acquisition, wireless control, and an intuitive mobile interface, Blynk significantly enhances the functionality and usability of the energy monitoring system.

This seamless integration brings the benefits of IoT technology into everyday life, promoting smarter and more sustainable energy consumption.

ThingSpeak

Go to [ThingSpeak](https://thingspeak.com) and click the “**Get Started For Free**” button to create a new account. This account is linked to a Mathworks account. So, if you already have a Mathworks account, you should log in with that account.

Creating New Channel

After your account is ready, sign in, open the “**Channels**” tab and select “**My Channels**”.

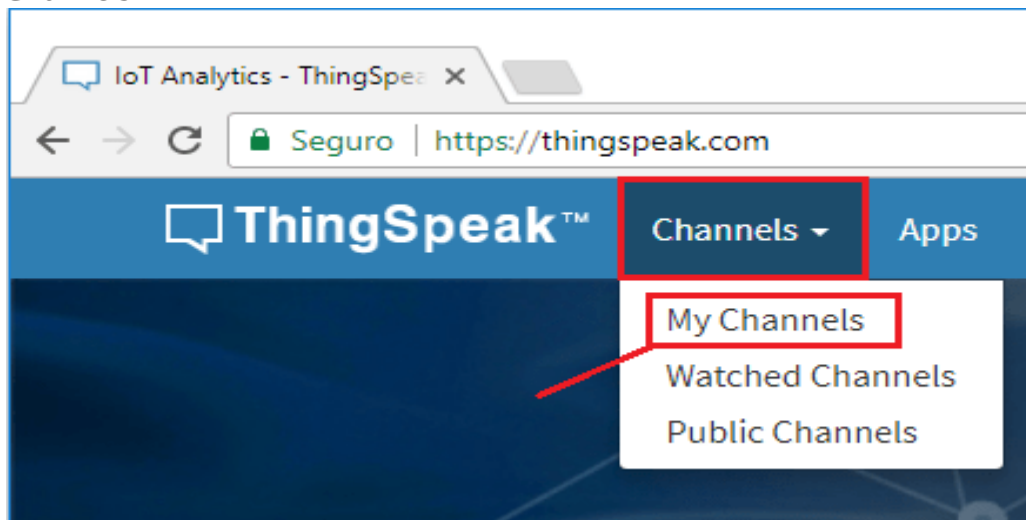
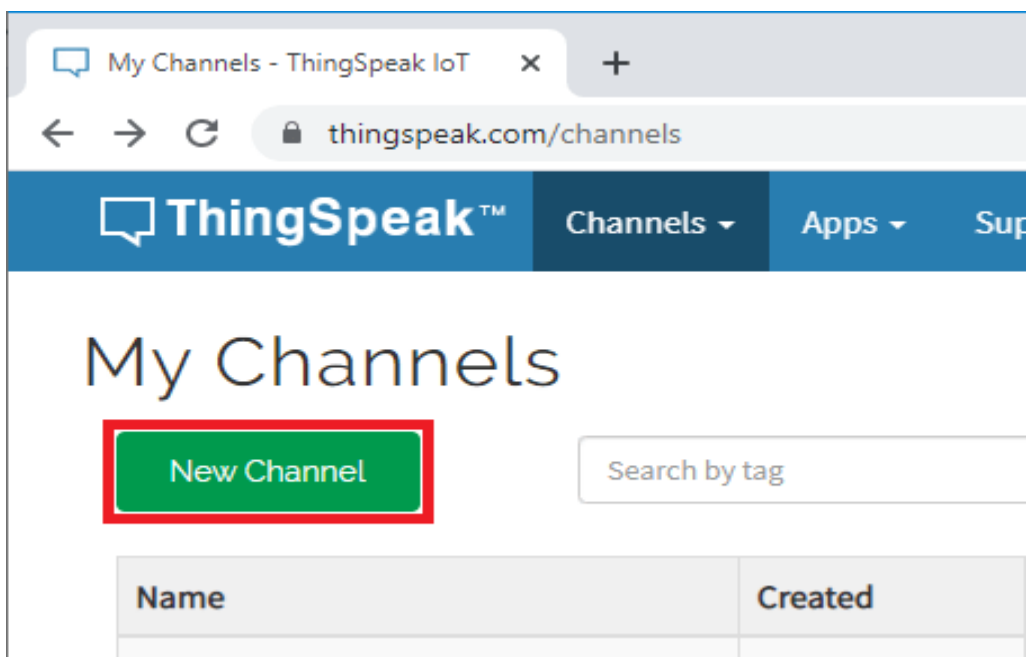


Fig. 5.12 Creating Channel in Thingspeak

Press the “New Channel” button to create a new channel.



Type a name for your channel and add a description. In this example, we'll just publish temperature. If you want to publish multiple readings (like humidity and pressure), you can enable more fields as you'll see later in this tutorial.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

ThingSpeak™

Channels ▾ Apps ▾ Devices ▾ Support ▾

Commercial Use How to Buy PM

Channel Settings

Percentage Complete 30%

Channel ID 2895799

Name ENERGY METER HOME AUTOMATION

Description

Field 1 voltage ☒

Field 2 current ☒

Field 3 power ☒

Field 4 energy ☒

Field 5 totalBill ☒

Field 6 billLoad1 ☒

Field 7 billLoad2 ☒

Field 8 billLoad3 ☒

Help

Channels store all the data that a ThingSpeak application collects. Each channel includes eight fields that can hold any type of data, plus three fields for location data and one for status data. Once you collect data in a channel, you can use ThingSpeak apps to analyze and visualize it.

Channel Settings

- **Percentage complete:** Calculated based on data entered into the various fields of a channel. Enter the name, description, location, URL, video, and tags to complete your channel.
- **Channel Name:** Enter a unique name for the ThingSpeak channel.
- **Description:** Enter a description of the ThingSpeak channel.
- **Field:** Check the box to enable the field, and enter a field name. Each ThingSpeak channel can have up to 8 fields.
- **Metadata:** Enter information about channel data, including JSON, XML, or CSV data.
- **Tags:** Enter keywords that identify the channel. Separate tags with commas.
- **Link to External Site:** If you have a website that contains information about your ThingSpeak channel, specify the URL.
- **Show Channel Location:**
 - **Latitude:** Specify the latitude position in decimal degrees. For example, the latitude of the city of London is 51.5072.
 - **Longitude:** Specify the longitude position in decimal degrees. For example, the longitude of the city of London is -0.1275.
 - **Elevation:** Specify the elevation position meters. For example, the elevation of the city of London is 35.052.

Fig.5.13 Adding the Fields

Click the **Save Channel** button to create and save your channel.



Customizing Chart
The chart can be customized, go to your **Private View** tab and click on the edit icon.

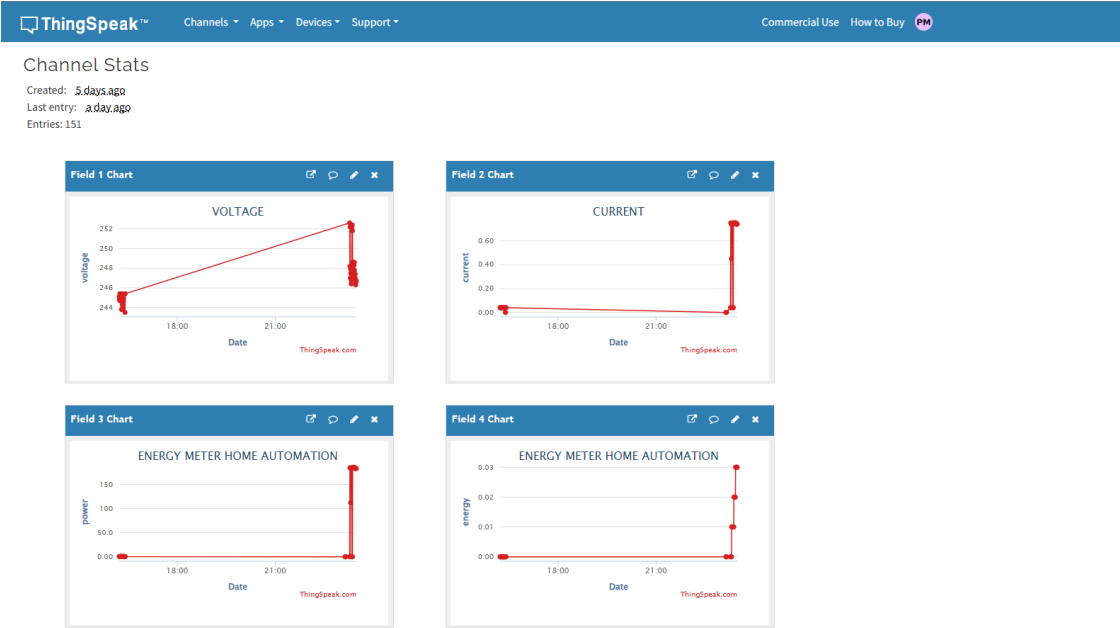


Fig .5.14 Creating the Channel Stats

You can give a title to your chart, customize the background color, x and y axis, and much more.

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

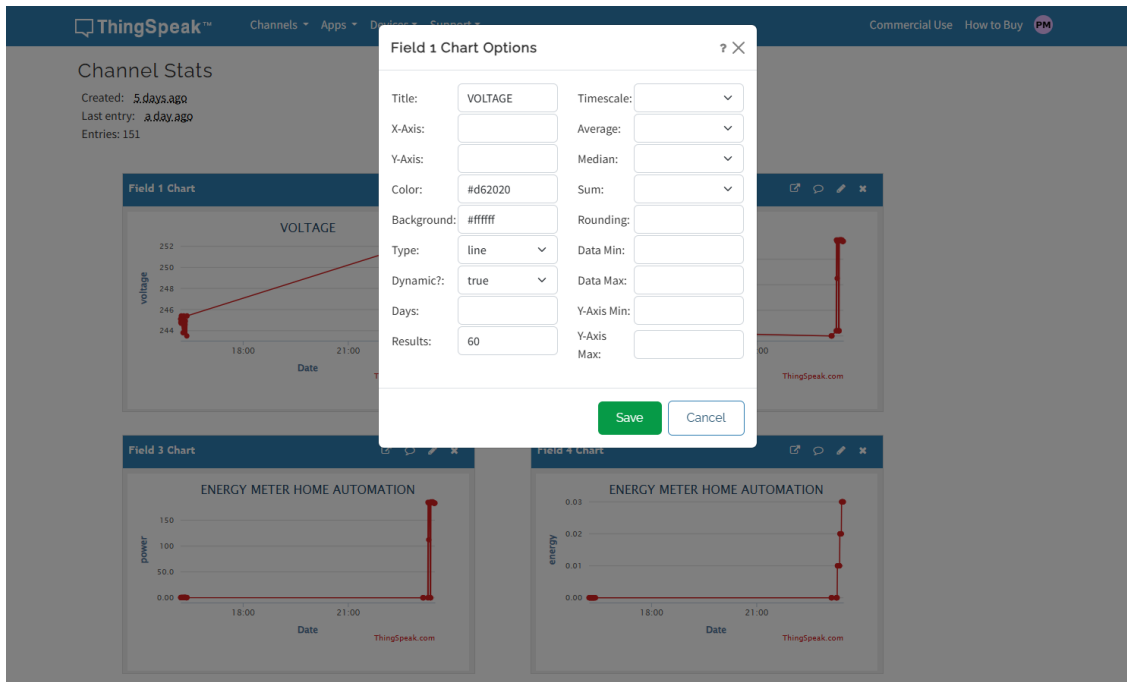


Fig 5.15 Customize the Chart Options

When you're done, press the "Save" button.

API Key

To send values from the ESP32 to ThingSpeak, you need the Write API Key. Open the "API Keys" tab and copy the Write API Key to a safe place because you'll need it in a moment.

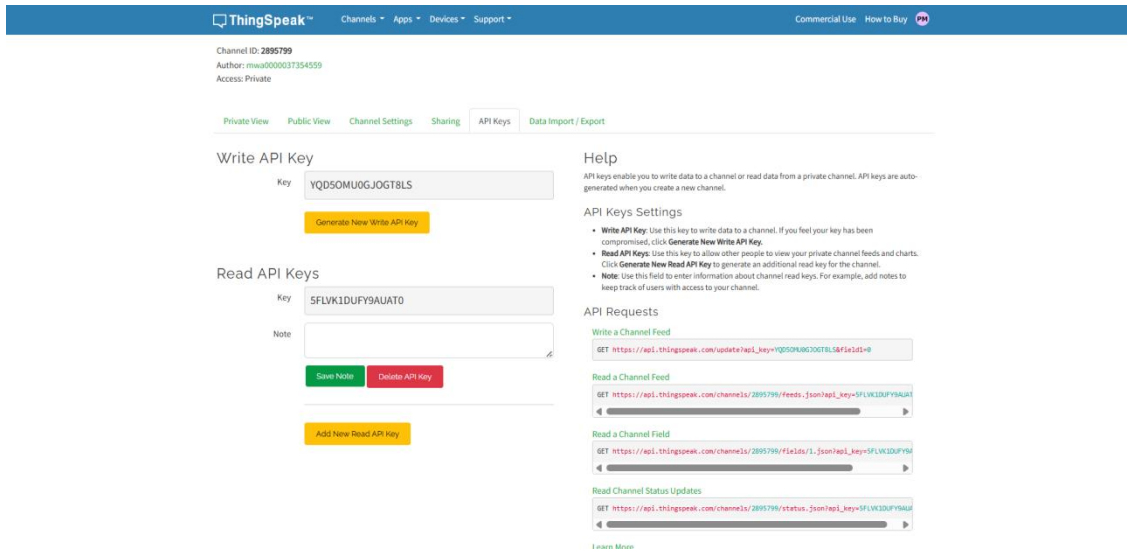


Fig.5.17 API key of Thingspeak

Go to "Data Import/Export"

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

- On your channel page, find and click on the **"Data Import/Export"** option (usually under the channel settings).

Select Export Options

- Choose either:
 - **"Export recent data"** (for the latest entries).
 - **"Export all data"** (for the entire dataset).

Download as Excel (CSV/XLSX)

- Select **CSV/XLSX** as the file format.
- Click **Download**, and your Excel file will be saved to your computer.

ThingSpeak™ Channels Apps Devices Support Commercial Use How to Buy PM

ENERGY METER HOME AUTOMATION

Channel ID: 2895799
Author: mwa000037354559
Access: Private

Private View Public View Channel Settings Sharing API Keys Data Import / Export

Import

Upload a CSV file to import data into this channel.

File No file chosen

Time Zone

Export

Download all of this Channel's feeds in CSV format.

Time Zone

Help

The correct format for data import is provided in this [CSV Import Template File](#). Use the field names *field1*, *field2*, and so on, instead of custom field names.

CSV Import Format

```
created_at,field1,field3,field4,field8,elevation
2019-01-01T10:11:12-05:00,11,33,44,88,10
```

Other Import and Export Options

You can also use MATLAB, the REST API, or the MQTT API to import and export channel data.

[Read Data](#)
[Write Data](#)

Fig. 5.18 Downloading the Excel Sheet contains data

Fig.5.19 Downloaded Excel Sheet

5.1 Project Codes

In this project we are describe entire project code,

```
#define BLYNK_TEMPLATE_ID "TMPL6mm0IsdC8"

#define BLYNK_TEMPLATE_NAME "Energy Meter With Home Automation"

#define BLYNK_AUTH_TOKEN "RFOIahTNiBIxsmIfPhWRKRPWK-K7bWmH"


#include <Arduino.h>

#include <Wire.h>

#include <LiquidCrystal_I2C.h>

#include <PZEM004Tv30.h>

#include <ACS712.h>

#include <WiFi.h>

#include <BlynkSimpleEsp32.h>

#include <HTTPClient.h>


// WiFi and Blynk credentials

char ssid[] = "Jaswaan";

char pass[] = "7995533479";


// ThingSpeak API Key

String apiKey = "YQD5OMU0GJOGT8LS";


// Sensors and modules

PZEM004Tv30 pzem(Serial2, 16, 17); // RX, TX for ESP32
```

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

```
ACS712 acs1(34, 3.3, 4095, 100);
```

```
ACS712 acs2(35, 3.3, 4095, 100);
```

```
ACS712 acs3(32, 3.3, 4095, 100);
```

```
ACS712 acs4(33, 3.3, 4095, 100);
```

```
LiquidCrystal_I2C lcd(0x27, 16, 2);
```

```
// Calibration factor for ACS712
```

```
int calibration_factor = 126;
```

```
// Relays
```

```
#define RELAY1 25
```

```
#define RELAY2 26
```

```
#define RELAY3 27
```

```
#define RELAY4 14
```

```
// Tariff per kWh
```

```
#define TARIFF 7.5
```

```
// Energy Consumption (Total over time)
```

```
float totalEnergyLoad1 = 0;
```

```
float totalEnergyLoad2 = 0;
```

```
float totalEnergyLoad3 = 0;
```

```
float totalEnergyLoad4 = 0;
```

```
// Relay Control via Blynk
```

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

```
BLYNK_WRITE(V11) { digitalWrite(RELAY1, param.asInt()); }
```

```
BLYNK_WRITE(V12) { digitalWrite(RELAY2, param.asInt()); }
```

```
BLYNK_WRITE(V13) { digitalWrite(RELAY3, param.asInt()); }
```

```
BLYNK_WRITE(V14) { digitalWrite(RELAY4, param.asInt()); }
```

```
void setup() {
```

```
  Serial.begin(115200);
```

```
  WiFi.begin(ssid, pass);
```

```
  while (WiFi.status() != WL_CONNECTED) {
```

```
    delay(1000);
```

```
    Serial.println("Connecting to WiFi...");
```

```
  }
```

```
  Serial.println("WiFi Connected!");
```

```
  Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);
```

```
  lcd.init();
```

```
  lcd.backlight();
```

```
  pinMode(RELAY1, OUTPUT);
```

```
  pinMode(RELAY2, OUTPUT);
```

```
  pinMode(RELAY3, OUTPUT);
```

```
  pinMode(RELAY4, OUTPUT);
```

```
  digitalWrite(RELAY1, LOW);
```

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

```
digitalWrite(RELAY2, LOW);

digitalWrite(RELAY3, LOW);

digitalWrite(RELAY4, LOW);

}

// Function to read AC current

float readCalibratedCurrent(ACS712 &sensor) {

    float average = 0;

    for (int i = 0; i < 100; i++) {

        average += sensor.mA_AC();

    }

    float mA = (abs(average / 100.0) - calibration_factor);

    if (mA <= 1) mA = 0;

    return mA / 1000.0; // Convert to Amps

}

void sendToThingSpeak(float voltage, float current, float power, float energy, float
totalBill, float billLoad1, float billLoad2, float billLoad3) {

    if (WiFi.status() == WL_CONNECTED) {

        HTTPClient http;

        String url = "http://api.thingspeak.com/update?api_key=" + apiKey +

            "&field1=" + String(voltage) +

            "&field2=" + String(current) +

            "&field3=" + String(power) +

            "&field4=" + String(energy) +
```

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

```
"&field5=" + String(totalBill) +
```

```
"&field6=" + String(billLoad1) +
```

```
"&field7=" + String(billLoad2) +
```

```
"&field8=" + String(billLoad3);
```

```
http.begin(url);
```

```
int httpResponseCode = http.GET();
```

```
if (httpResponseCode > 0) {
```

```
    Serial.println("Data sent to ThingSpeak successfully.");
```

```
} else {
```

```
    Serial.print("Error sending data: ");
```

```
    Serial.println(httpResponseCode);
```

```
}
```

```
http.end();
```

```
}
```

```
}
```

```
void loop() {
```

```
    Blynk.run();
```

```
    float voltage = pzem.voltage();
```

```
    float current = pzem.current();
```

```
    float power = pzem.power();
```

```
    float energy = pzem.energy(); // Total energy from PZEM
```

```
    float frequency = pzem.frequency();
```

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

```
float pf = pzem.pf();
```

```
float currentLoad1 = readCalibratedCurrent(acs1);
```

```
float currentLoad2 = readCalibratedCurrent(acs2);
```

```
float currentLoad3 = readCalibratedCurrent(acs3);
```

```
float currentLoad4 = readCalibratedCurrent(acs4);
```

```
float totalCurrent = currentLoad1 + currentLoad2 + currentLoad3 + currentLoad4;
```

```
// Power for each load ( $P = V * I$ )
```

```
float powerLoad1 = voltage * currentLoad1;
```

```
float powerLoad2 = voltage * currentLoad2;
```

```
float powerLoad3 = voltage * currentLoad3;
```

```
float powerLoad4 = voltage * currentLoad4;
```

```
// Energy per load in 5s interval ( $E = P * t$ , with t in hours)
```

```
float energyLoad1 = (powerLoad1 * 5) / (1000 * 3600);
```

```
float energyLoad2 = (powerLoad2 * 5) / (1000 * 3600);
```

```
float energyLoad3 = (powerLoad3 * 5) / (1000 * 3600);
```

```
float energyLoad4 = (powerLoad4 * 5) / (1000 * 3600);
```

```
// Accumulate total energy consumption per load
```

```
totalEnergyLoad1 += energyLoad1;
```

```
totalEnergyLoad2 += energyLoad2;
```

```
totalEnergyLoad3 += energyLoad3;
```

```
totalEnergyLoad4 += energyLoad4;
```

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

```
// Calculate the correct bill per load

float billLoad1 = totalEnergyLoad1 * TARIFF;

float billLoad2 = totalEnergyLoad2 * TARIFF;

float billLoad3 = totalEnergyLoad3 * TARIFF;

float billLoad4 = totalEnergyLoad4 * TARIFF;


// Calculate total bill

float totalBill = billLoad1 + billLoad2 + billLoad3 + billLoad4;


// Display on LCD

lcd.clear();

lcd.setCursor(0, 0);

lcd.print("V: "); lcd.print(voltage);

lcd.print(" I: "); lcd.print(current);


lcd.setCursor(0, 1);

lcd.print("B: "); lcd.print(totalBill);


// Send data to Blynk

Blynk.virtualWrite(V0, voltage);

Blynk.virtualWrite(V1, current);

Blynk.virtualWrite(V2, power);

Blynk.virtualWrite(V3, energy);

Blynk.virtualWrite(V4, frequency);
```

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

```
Blynk.virtualWrite(V5, pf);

Blynk.virtualWrite(V6, totalBill);

Blynk.virtualWrite(V7, billLoad1);

Blynk.virtualWrite(V8, billLoad2);

Blynk.virtualWrite(V9, billLoad3);

Blynk.virtualWrite(V10, billLoad4);


// Send individual current readings to Blynk

Blynk.virtualWrite(V15, currentLoad1);

Blynk.virtualWrite(V16, currentLoad2);

Blynk.virtualWrite(V17, currentLoad3);

Blynk.virtualWrite(V18, currentLoad4);

Blynk.virtualWrite(V19, totalCurrent);


// Send data to ThingSpeak

sendToThingSpeak(voltage, current, power, energy, totalBill, billLoad1, billLoad2,
billLoad3);

delay(15000); // 15-second delay for ThingSpeak update
}
```

CHAPTER 5

Testing and Result Analysis

5.1 Test case 1:

1. When All the Loads are off without load connection

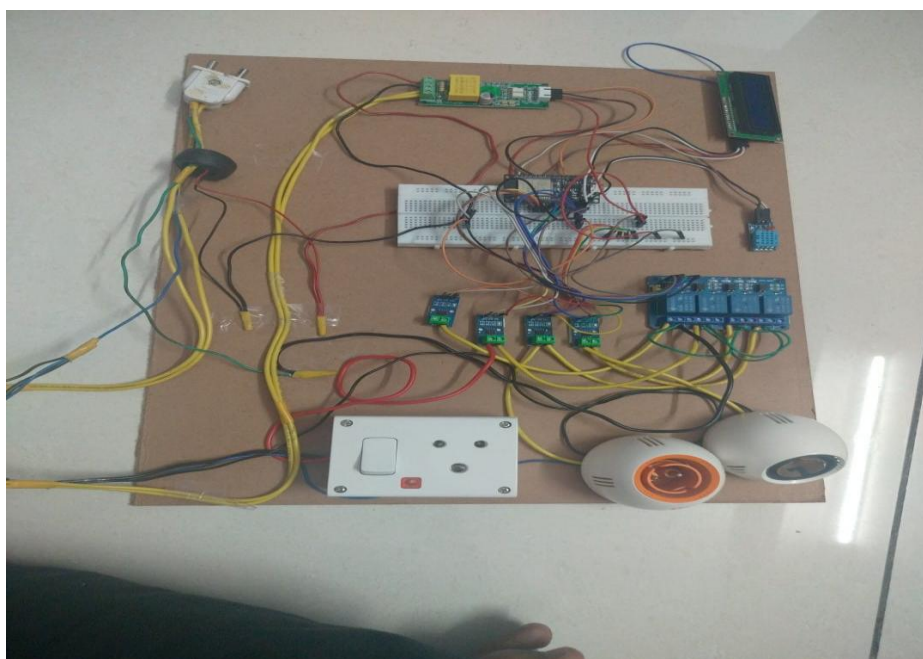


Fig. 5.1 All Loads are off and not connect

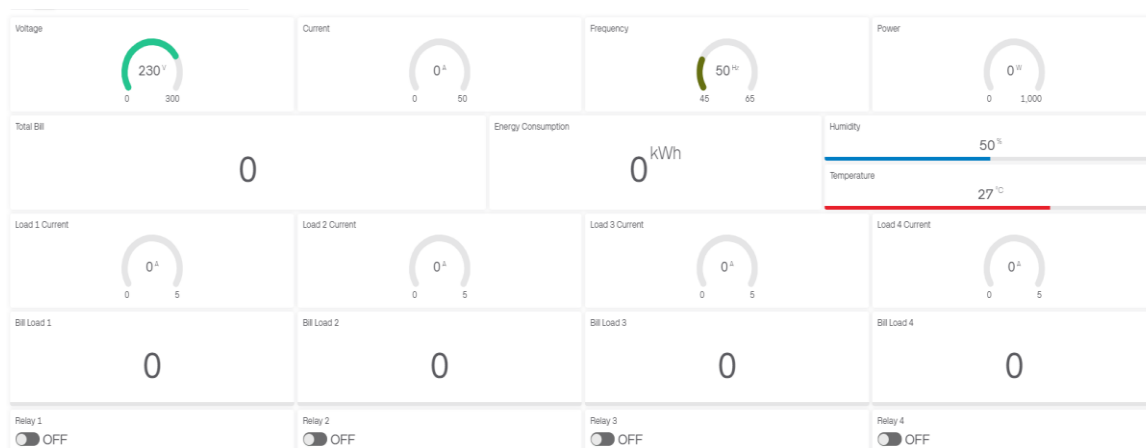


Fig 5.2 Blynk Dashboard without any load

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

5.2 Test case 2

When Load 1 is in ON condition

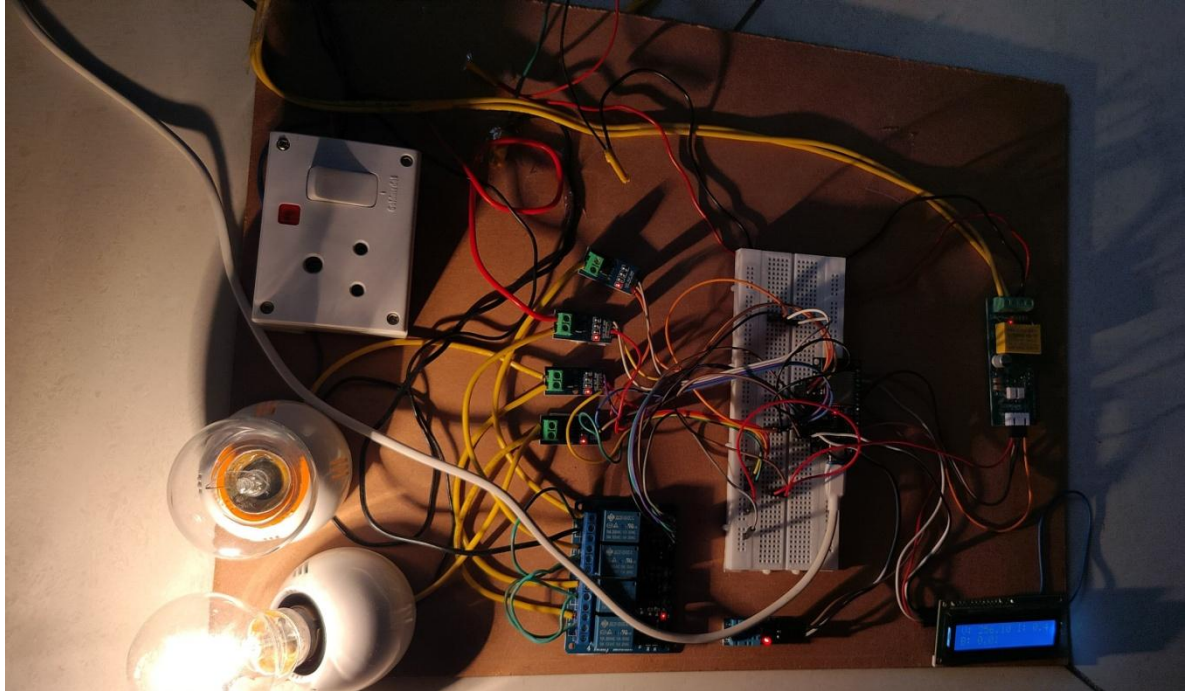


Fig 5.3 When Load 1 is in ON Condition

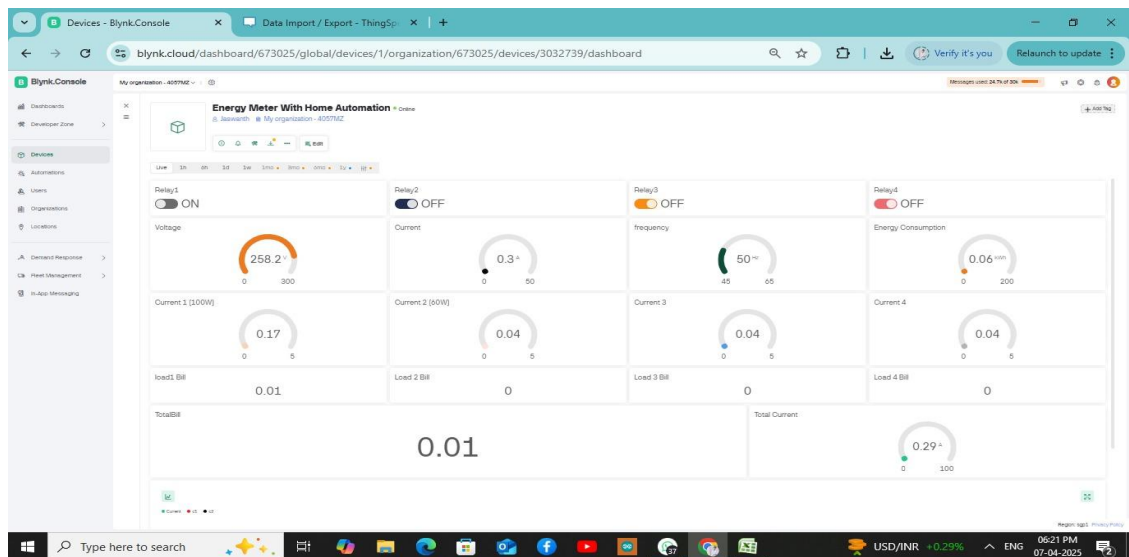


Fig.5.4 Blynk Dashboard when Load 1 is ON

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

5.3 Test case 3

When Load 2 is in ON condition

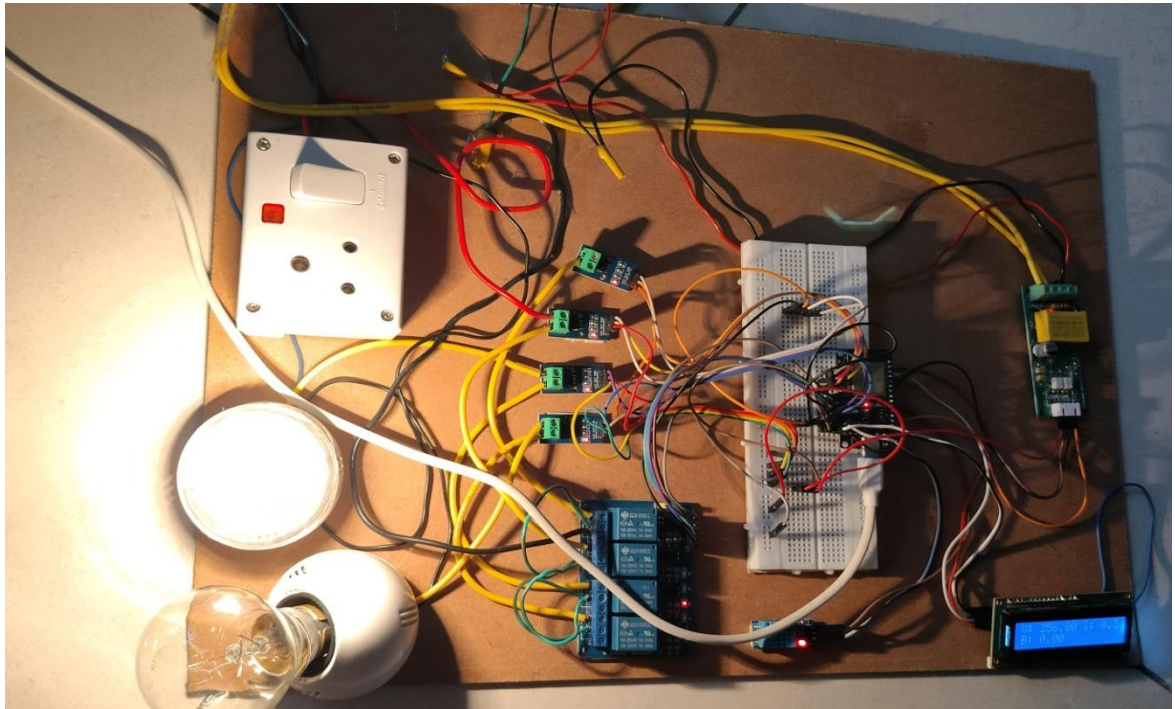


Fig.5.5 When Load 2 is in ON condition

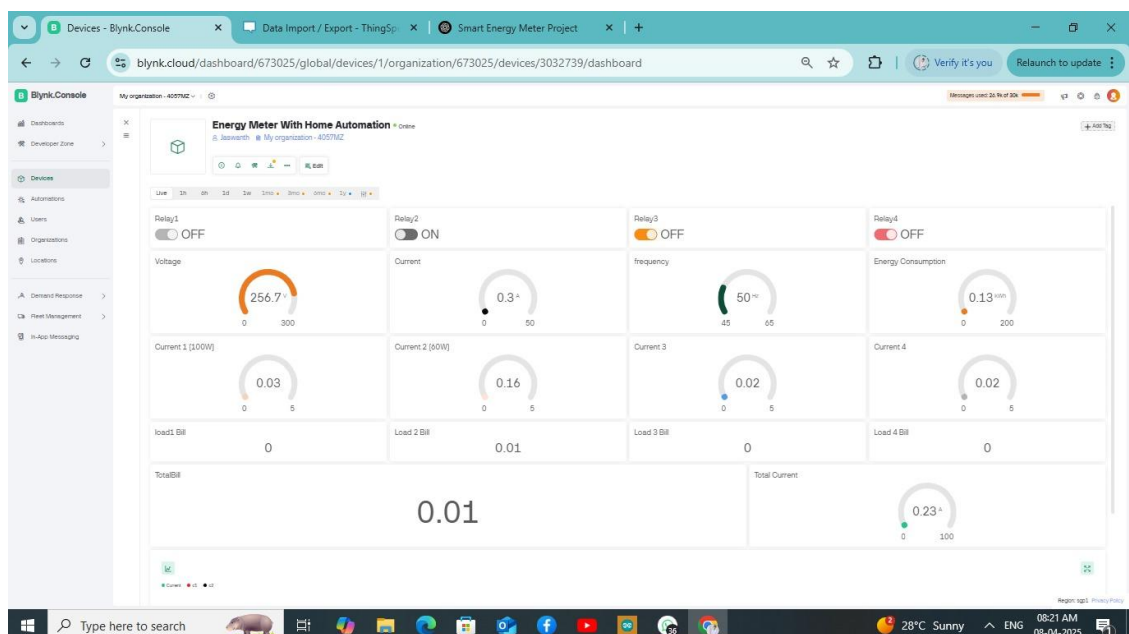


Fig 5.6 Blynk Dashboard when Load 2 in ON condition

SMART ENERGY METER WITH REALTIME MONITORING AND PREDICTIVE BILL ANALYSIS

5.4 Test case 4

When Load 3 is connected

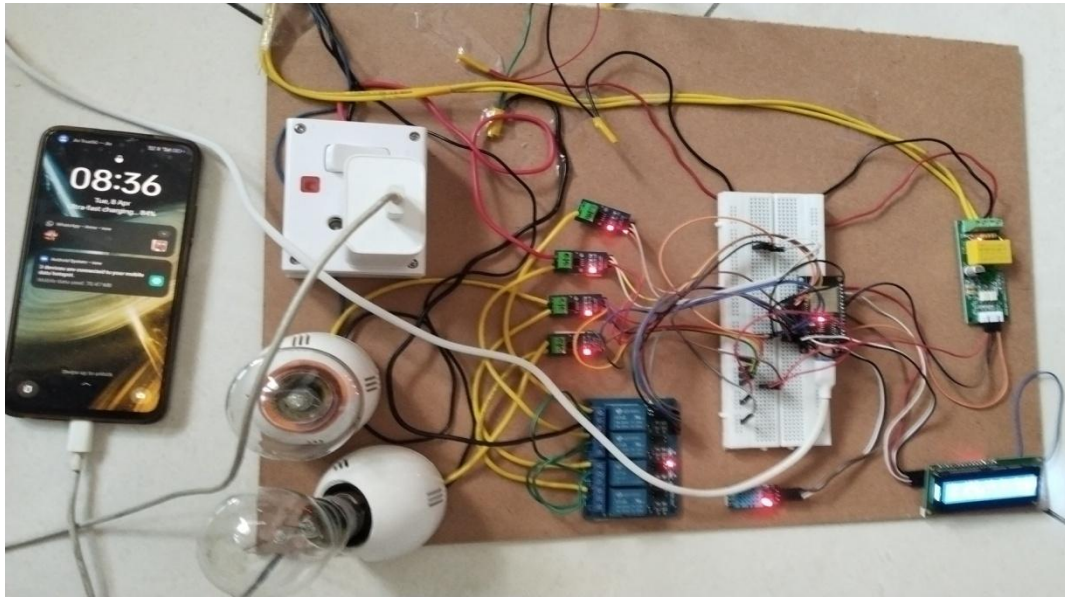


Fig 5.7 When Load 3 is in ON condition

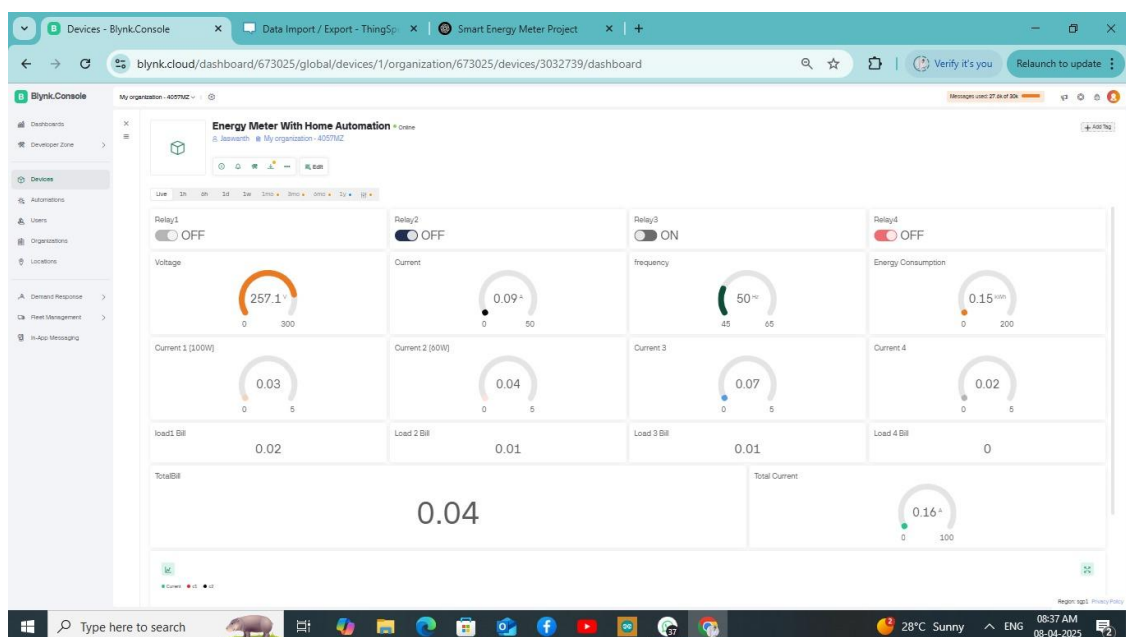


Fig 5.8 Blynk Dash Board when Load 3 is ON

5.5 Test case 5

Conclusion and Future Scope

Conclusion

The Smart Energy Meter project successfully achieved its goals of **real-time monitoring, predictive bill analysis, and remote load control**. The integration of IoT technology through **Blynk** added flexibility and enhanced user interaction with the system.

The system provides an efficient way for users to monitor energy consumption, control home appliances remotely, and adopt energy-saving habits based on real-time feedback.

The project demonstrates a low-cost, scalable solution for smart energy management at the household and industrial levels.

Future Scope

Although the current system fulfills its basic requirements, several improvements and expansions can be incorporated in the future:

- **Mobile Notifications:**
Adding push notifications through Blynk when voltage, current, or energy thresholds are exceeded.
- **Data Logging:**
Storing historical consumption data in the cloud for detailed energy usage reports.
- **Voice Assistant Integration:**
Connecting the system with Google Assistant, Alexa, or Siri for voice-controlled appliance management.
- **Solar Power Monitoring:**
Integrating solar panels and monitoring their power generation to promote the use of renewable energy sources.
- **Energy Saving Suggestions:**
Implementing AI-based algorithms to analyze energy consumption patterns and suggest ways to reduce electricity bills.
- **Three-Phase System Expansion:**
Modifying the system to monitor three-phase industrial loads.



KKR & KSR INSTITUTE OF TECHNOLOGY AND SCIENCES

(Autonomous)

Accredited by NBA & NAAC with Grade "A" and Affiliated to JNTUK-Kakinada
Vinjanampadu, Vatticherukuru Mandal, Guntur, Andhra Pradesh 522017

DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING

PROJECT OR SCANNER FORM

Academic Year: 2024-2025

Batch: 10

NAME OF THE GUIDE: Mrs. SD. Vaheedha

BATCH MEMBERS-

SIGNATURE

1. **DAMARAVARAPU NAGA JASWANTH (22JR5A0406)**
2. **EKA. P. L. NARASIMHA (21JR1A0459)**
3. **BANISSETTY AAKASH (21JR1A0436)**
4. **B ASHOK BABU (21JR1A0446)**

SIGNATURE OF GUIDE

SIGNATURE OF HOD